

CognitiveOC v3 — Complete User and Developer Guide

Edition: Zero-Risk Corpus + Training Control + Operator Manual

Status: 80 Python modules · 88 packaged files · All syntax-clean · All CLI commands verified

Hardware target: RTX 5060 8GB · AMD NPU · 32 GB RAM · 1 TB corpus SSD

Sections: 14 core sections + 12 operator manual sections (A-L) · 3715 lines

Run all commands from: `<repo_root>/cognitiveoc_v3/` unless stated otherwise

Table of Contents

Core Guide (Sections 1-14)

1. [Overview](#)
2. [Architecture](#)
3. [Corpus Engineering](#)
4. [Dataset Generation](#)
5. [UI Startup and Usage](#)
6. [Training](#)
7. [Observability](#)
8. [Memory · Retrieval · KG · Reasoning · Research · Validation](#)
9. [CLI Reference](#)
10. [File and Folder Reference](#)
11. [Startup and Shutdown](#)
12. [Troubleshooting](#)
13. [End-to-End Flow](#)
14. [Frozen vs Planned](#)

Operator Manual (Sections A-L)

- [A. Corpus Engineering Handbook](#)
- [B. Warehouse Operations Manual](#)
- [C. Source Acquisition Manual](#)
- [D. Licensing and Governance Manual](#)
- [E. Corpus Quality System](#)
- [F. Release Engineering Manual](#)
- [G. Training Governance Manual](#)
- [H. Segmented Training Playbook](#)
- [I. Corpus Target Strategy](#)
- [J. Complete CLI Reference \(Updated\)](#)

- K. Full End-to-End Story
 - L. Project Status Matrix
-

1. Overview

What CognitiveOC is

CognitiveOC (COC) is a **local-first cognitive orchestration core** — a complete AI system that runs entirely on your own hardware with no cloud dependencies at runtime.

It is not a plain chatbot. It is a layered cognitive system that combines:

- A **700M-parameter foundation model** trained on governed, curated data
- A **multi-type memory system** that remembers across sessions
- A **hybrid retrieval engine** that grounds answers in indexed documents
- A **SQLite knowledge graph** with 500 K triple capacity
- A **chain-of-thought reasoning engine**
- A **multi-loop research engine** with validation
- A **9-module human cognition layer** (emotion, intent, teaching level, goal tracking)
- A **2-tier guardrail framework** (hard integrity + 12 toggleable cognitive guards)
- A **workflow engine** for multi-step automated tasks
- A **dataset generation engine** that captures training examples from live use
- A **governed corpus pipeline** with 12 pipeline stages, release locking, and full audit
- A **training control system** with shard tracking, ledger, resume guard, and provenance
- A **native PySide6 desktop app** (primary interface)
- A **FastAPI-based web app** (secondary interface, port 8765)

Why this architecture exists

Each component exists to solve a specific problem:

Problem	Solution
Model forgets between sessions	8-type memory system (SQLite)
Hallucinated facts	Hybrid retrieval + fact validation
Unstructured knowledge	SQLite knowledge graph
No step-by-step reasoning	Chain-of-thought reasoning engine
Generic responses	Human cognition layer (emotion, teaching level)
Safety	2-tier guardrail framework
Dependency on cloud APIs	Local 700M decoder on GPU
Noisy training data	12-stage governed corpus pipeline
Data loss on restart	Shard tracker + training ledger

Current baseline vs v3 target

Item	Baseline (verified)	v3 Target
Decoder	700M parameters	Pre-trained on 30B token release
Tokenizer	48K vocab, SentencePiece	Trained on v1 release
Memory	8 types, SQLite	Production-ready
Retrieval	BM25 + BGE-768	Production-ready
KG	500K triples, FTS5	Production-ready
Corpus	None	30B token v1 release
Training	Training loop verified	Requires tokenizer + release
Desktop UI	PySide6, all panels	Production-ready
Web UI	18 endpoints, SSE	Production-ready

What is complete now: Repository, all subsystem code, corpus pipeline, training control, CLI, both UIs, all governance files.

What is not done yet: Source acquisition, release build, tokenizer training, model training. These are operational tasks, not code tasks.

2. Architecture

Runtime flow (per request)



Compute map

Component	Device	VRAM/ RAM
700M decoder (inference)	GPU · RTX 5060 8 GB · bf16	2.10 GB
700M decoder (training)	GPU · gradient checkpointing	7.80 GB
13 encoders (BGE, intent, emotion...)	NPU → CPU fallback	~730 MB
Memory / KG / retrieval	CPU + SSD	SQLite, FAISS-lite

Component	Device	VRAM / RAM
Corpus pipeline	CPU + 1 TB SSD	All I/O-bound
Web UI server	CPU	Negligible
Desktop UI	CPU	Qt event loop

Context window budget (8 192 tokens)

Slot	Tokens
System prompt	256
Memory recall	512
Retrieval chunks	2 048
KG triples	256
Tool results	512
Conversation history	512
Cognition injection	128
Generation headroom	1 024
Total used	5 248 · 2 944 margin

What must never be casually edited

models/transformer.py	← 700M architecture – any change breaks checkpoints
cognition/cognition.py	← isolation guarantee – must survive toggling
safety/guardrails.py	← safety contract – hard guards must stay on
memory/memory.py	← SQLite schema – changes corrupt stored memory
retrieval/rag.py	← index format – changes break existing indexes
knowledge/graph.py	← triple schema – changes corrupt the KG
inference/generator.py	← must match decoder exactly
eval/run_suite.py	← benchmark comparability
train/train_model.py	← only extend, never rewrite
data/pipeline.py	← corpus modules call this as a base
tokenizer/tokenizer.py	← any change requires full retraining

What is safe to extend

config.py	← add keys, never remove existing ones
-----------	--

main.py	← add CLI commands, never remove existing ones
corpus/*	← new source cleaners, scorers, validators
governance/*	← JSON data files, append-only
audit/*	← new report types
release/*	← new verification checks
train/training_ledger.py	← new session fields
train/shard_tracker.py	← new shard metadata
train/resume_guard.py	← add checks, never remove existing ones
train/provenance.py	← add fields, never remove existing ones
dataset/generator.py	← new export types

3. Corpus Engineering

What the corpus system is

The corpus system is a **governed data pipeline**, not a folder of text files. It enforces quality, provenance, and reproducibility at every step and prevents any unreviewed data from reaching the model.

Checklist

- ✓ What it is: 12-stage pipeline with audit, quality gates, release locking
- ✓ Where it lives: `corpus/`, `governance/`, `release/`, `audit/`
- ✓ How to start it: `python main.py corpus <subcommand>`
- ✓ What commands are used: see §9 CLI Reference
- ✓ What files are involved: `governance/source_registry.json`, warehouse SSD
- ✓ What outputs are produced: cleaned text, manifests, releases, audit logs
- ✓ What can fail: licence mismatch, checksum fail, missing raw files
- ✓ How to troubleshoot: see §12

Two separate storage systems

Corpus Warehouse — lives on the 1 TB SSD (path set in `config.py`):

```

D:/corpus_warehouse/      ← set CORPUS_WAREHOUSE_DIR in config.py
├─ raw/                   ← immutable source archives, never modified
│   ├── books/
│   ├── educational/
│   ├── reasoning/
│   ├── conversations/
│   ├── technical_docs/
│   ├── articles/
│   ├── research_papers/
│   ├── synthetic/
│   ├── cognition/
│   ├── retrieval/
│   └─ kg/
├─ cleaned/               ← post-pipeline UTF-8 paragraphs
├─ deduplicated/          ← after cross-source MinHash dedup
├─ scored/                ← paragraph score metadata
├─ review_queue/          ← pending.jsonl · approved.jsonl ·
rejected.jsonl
├─ approved/              ← quality-gated text ready for release
├─ rejected/              ← failed content (kept for audit, never used)
├─ synthetic/             ← COC-generated data (versioned)
│   └─ v1/
├─ manifests/             ← per-source manifest JSON files
├─ releases/              ← assembled training releases
│   └─ v1/
│       ├── train.txt
│       ├── val.txt
│       ├── test.txt
│       ├── manifest.json
│       ├── checksums.sha256
│       └─ LOCK            ← written by lock-release
├─ governance_logs/       ← daily audit JSONL shards
└─ archive/               ← recalled releases, retired sources

```

Repository — inside `cognitiveoc_v3/`:

```

governance/
├─ source_registry.json    ← all registered sources (version-tracked)
├─ license_rules.json      ← 17 licence types with risk scores
└─ approval_log.jsonl     ← append-only approval history (committed)

```

Nothing from the warehouse belongs inside the repository. The repository stores metadata and code only.

Stage 1	Acquire	Download/copy to warehouse/raw/<category>/
Stage 2	Validate	Licence check + content review → registry update
Stage 3	Normalize	PDF/EPUB/HTML/JSONL → UTF-8 plain text
Stage 4	Clean reconstruction	Boilerplate removal, PII scan, paragraph
Stage 5	Deduplicate	Within-source (cosine 0.92) + cross-source (MinHash 0.85)
Stage 6	Score	Quality score + category score + risk score per paragraph
Stage 7	Review	Auto-approve ≥0.70 quality + ≤0.20 risk; else human queue
Stage 8	Approve	Source locked as approved; registry updated
Stage 9	Split	90 % train / 5 % val / 5 % test, seed=42
Stage 10	Release checksums.sha256	Assemble files, generate manifest.json +
Stage 11	Audit	All events → governance_logs/audit_YYYYMMDD.jsonl
Stage 12	Archive	Raw files → warehouse/archive/ after release signed

Quality gates

Quality score	Risk score	Decision
≥ 0.70	≤ 0.20	Auto-approve
0.45 - 0.70	0.20 - 0.50	Human review queue
< 0.45	any	Auto-reject
any	> 0.50	Auto-reject
Synthetic (any score)	—	Always human review

Licence risk matrix (summary)

Licence	Risk	Training use
Public Domain / CC0	0.0	YES
Apache 2.0 / MIT / BSD	0.1	YES
CC-BY 4.0	0.1	YES (attribute in manifest)
CC-BY-SA 4.0	0.2	YES (note SA in manifest)
CC-BY-NC 4.0	0.3	YES for non-commercial use

Licence	Risk	Training use
CC-BY-ND	0.5	Human review required
Unknown / unlicensed	0.8	Human review required
All Rights Reserved	1.0	HARD REJECT
ToS violation	1.0	HARD REJECT

Full matrix: `governance/license_rules.json`

Category-by-category source strategy

Category A — Books (public domain)

Item	Detail
Purpose	Long-form coherence, vocabulary depth, narrative structure
Best sources	Project Gutenberg (PD), Standard Ebooks (CC0), Wikisource (CC-BY-SA)
Acceptable	Internet Archive text (older OCR — extra cleaning needed)
Rejected	Post-1927 copyright books, Google Books snippets, Z-Library, Anna's Archive
Quality	High (Standard Ebooks), Medium-High (raw Gutenberg — OCR artifacts)
Licence risk	0.0 (PD/CC0)
v1 token target	8 B tokens
Warehouse ceiling	30 B tokens
Cleanup	Strip Gutenberg header/footer (regex), OCR artifact removal, paragraph reconstruction
Split	90/5/5 standard
In v1 release	YES
Synthetic later	NO — real books are better

Category B — Educational Content

Item	Detail
Purpose	Structured explanation, worked examples, concept definitions
Best sources	OpenStax (CC-BY), CK-12 (CC-BY-NC), MIT OCW text (CC-BY-NC-SA)
Acceptable	Wikibooks English (CC-BY-SA), Khan Academy transcripts (CC-BY-NC-SA)
Rejected	Coursera/edX scrapes (ToS violation), copyrighted textbook PDFs
Licence risk	0.1 – 0.4
v1 token target	4 B tokens
In v1 release	YES
Synthetic later	YES — COC-generated explanations of its own subsystems

Category C — Reasoning / STEM

Item	Detail
Purpose	Step-by-step reasoning, mathematical thinking, scientific analysis
Best sources	NuminaMath/OpenR1 (Apache 2.0), GSM8K (MIT), ARC (CC-BY), MATH (MIT), LogiQA (MIT)
Acceptable	MMLU training splits (CC-BY), SciQ (CC-BY)
Rejected	BIG-Bench raw dumps (too noisy), HELM benchmark data (evaluation-restricted)
Format	Convert to <code><reasoning>...</reasoning></code> COC special-token format
Licence risk	0.1
v1 token target	4 B tokens
In v1 release	YES — high priority
Synthetic later	YES — COC workflow engine generates reasoning chains

Category D — Conversations / Instruction

Item	Detail
Purpose	Instruction-following, dialogue, QA
Best sources	Dolly 15k (CC-BY-SA 3.0), OASST2 (Apache 2.0), FLAN formatted (Apache 2.0)
Acceptable	Reviewed Alpaca (CC-BY-NC)
Rejected	GPT-3.5/4 outputs as training data (OpenAI ToS), UltraChat, ChatGPT dumps
Hard rule	Response generator must be open-source or human-authored
Format	<code><user>...</user><assistant>...</assistant></code>
Licence risk	0.1 - 0.3
v1 token target	3 B tokens
Warehouse-only	ShareGPT (pending licence review)
Synthetic later	YES — best synthetic category; COC's own conversations

Category E — Technical Documentation

Item	Detail
Purpose	Precise technical writing, factual accuracy for CS/ML domains
Best sources	Python docs (PSF), PyTorch docs (BSD-3), HuggingFace docs (Apache), Wikipedia CS/math/science (CC-BY-SA)
Acceptable	MDN Web Docs (CC-BY-SA)
Rejected	AWS/Azure/GCP proprietary docs (ToS), Stack Overflow at scale (attribution complexity)
Wikipedia	Article text only; filter to CS, math, science, linguistics; skip stubs < 500 words
v1 token target	3 B tokens
In v1 release	YES

Category F — Long-Form Articles

Item	Detail
Purpose	Sustained coherence, argument development, factual grounding
Best sources	The Conversation (CC-BY-ND — training use OK), ArXiv intros (CC-BY), Wikinews (CC-BY 2.5)
Rejected	News agency content (Reuters, AP, BBC — ToS), SEO blogs, BuzzFeed/listicles
v1 token target	3 B tokens
In v1 release	YES

Category G — Research Papers

Item	Detail
Purpose	Highest-quality technical reasoning and scientific analysis
Best sources	ArXiv CC-BY papers (full text), ACL Anthology (CC-BY), PubMed OA (CC-BY)
Acceptable	S2ORC (CC-BY-NC — warehouse only due to NC clause), CORE (CC-BY papers only)
Rejected	Elsevier/Springer/Wiley without CC licence, SciHub-sourced papers
Filter	Min 5 citations for papers > 2 years old; strip reference lists, equations
v1 token target	3 B tokens
Warehouse-only	S2ORC (NC clause — do not include in released checkpoint without confirming non-commercial)

Category H — COC Synthetic Data

Item	Detail
Purpose	Train COC specifically on its own operational patterns
Source	<code>dataset/generator.py</code> — live session capture
Types	conversation, retrieval, kg, memory, teaching, emotion, reasoning
Hard rules	Never auto-approved; always human review; versioned separately; never mixed with raw data
Quality bar	Min 0.70 (higher than real data)
v1 token target	1 B tokens (reviewed)
Synthetic later	YES — grows with each live session

Category I — Human Cognition

Item	Detail
Purpose	Supports the 9-module Human Cognition Layer
Best sources	PG psychology classics (PD), OpenStax Psychology (CC-BY), CogSci papers (CC-BY), PubMed neuroscience (CC-BY)
Rejected	Popular psychology blogs, self-help content
v1 token target	1 B tokens

Category J — Retrieval Material

Item	Detail
Purpose	Trains retrieval-grounded response patterns
Best sources	MS MARCO (MIT), Natural Questions (CC-BY-SA), TriviaQA (Apache), HotpotQA (CC-BY-SA)
Format	<code><retrieval>...</retrieval></code> COC special-token format
v1 token target	0.5 B tokens

Category K — Knowledge Graph Material

Item	Detail
Purpose	Trains structured factual generation and triple extraction
Best sources	DBpedia abstracts (CC-BY-SA), ConceptNet (CC-BY-SA), Wikidata text (CC0)
Format	<code><kg>...</kg></code> COC special-token format
v1 token target	0.5 B tokens

v1 Release summary

Category	Tokens	Key sources
A — Books	8 B	Project Gutenberg, Standard Ebooks
B — Educational	4 B	OpenStax, CK-12
C — Reasoning/STEM	4 B	NuminaMath, GSM8K, ARC, MATH
D — Conversations	3 B	Dolly 15k, OASST2, FLAN
E — Technical docs	3 B	Wikipedia CS/math, PyTorch/HF docs
F — Long-form	3 B	ArXiv intros, The Conversation
G — Research papers	3 B	ArXiv CC-BY, ACL Anthology
H — COC synthetic	1 B	COC generator (reviewed)
I — Cognition	1 B	OpenStax Psychology, CogSci papers
J — Retrieval	0.5 B	MS MARCO, NQ, TriviaQA
K — KG material	0.5 B	DBpedia, ConceptNet
Total	31 B	

Corpus commands — step-by-step

Working directory: `cognitiveoc_v3/`

First-time setup (once)

```
bash
```

```
# 1. Set your 1TB SSD path in config.py:
#     CORPUS_WAREHOUSE_DIR = Path("D:/corpus_warehouse")    ← change this

# 2. Verify the system is live
python main.py corpus warehouse-stats
# Expected: all categories at 0 tokens, no errors

python main.py corpus audit-report
# Expected: empty report, no errors
```

Register a source

```
bash

python main.py corpus register-source A-gutenberg-20260701 \
  --category A \
  --name "Project Gutenberg English" \
  --url https://gutenberg.org \
  --licence "Public Domain" \
  --licence-id pd \
  --licence-risk 0.0 \
  --raw-path "D:/corpus_warehouse/raw/books/gutenberg/" \
  --raw-sha256 "" \
  --operator mpssp \
  --notes "Main PD English fiction and non-fiction corpus"

# Output: ✓ Source registered: A-gutenberg-20260701
```

Validate a source (after licence + content review)

```
bash

python main.py corpus validate-source A-gutenberg-20260701 --operator mpssp

# To reject instead:
python main.py corpus validate-source A-gutenberg-20260701 \
  --reject --reason "OCR quality too low" --operator mpssp
```

Run the pipeline

```
bash
```

```
# Place .txt files in D:/corpus_warehouse/raw/books/gutenberg/ first, then:
python main.py corpus run-pipeline A-gutenberg-20260701 \
  --stages clean,dedup,score \
  --operator mpssp \
  --verbose

# Output shows:
#   [clean]  N paragraphs written
#   [dedup]  N → M paragraphs (X removed)
#   [score]  auto-approved: N   human-queue: M   auto-rejected: K
```

Review borderline items

```
bash

python main.py corpus review --source A-gutenberg-20260701 --operator mpssp
# Controls: [a]pprove [r]eject [s]kip [q]uit

# Check queue stats without entering review:
python main.py corpus review --stats
```

Build and lock the release

```
bash
```



```
# Dry run first – no files written
python main.py corpus build-release v1 --dry-run

# Build for real
python main.py corpus build-release v1 \
  --categories A,B,C,D,E,F,G,H,I,J,K \
  --token-budget 300000000000

# Verify checksums and leakage
python main.py corpus verify-release v1

# Sign (human approval)
python main.py corpus sign-release v1 --operator mpssp

# Lock (immutable for training)
python main.py lock-release v1 \
  --operator mpssp \
  --notes "Phase 1 pre-training start"

# List all releases
python main.py corpus list-releases
```

Check warehouse progress

```
bash

python main.py corpus warehouse-stats
# Shows: tokens by category, GB used, % toward v1 and warehouse targets

python main.py corpus audit-report
python main.py corpus audit-report --source A-gutenberg-20260701
python main.py corpus audit-report --from 2026-07-01 --to 2026-07-31
```

4. Dataset Generation

What it is

The dataset generator captures feedback, errors, and examples from live COC sessions and exports them as training-ready datasets. It is the source of Category H corpus data (COC synthetic).

Critical rule: `auto_approve = False` in all cases. Nothing is ever automatically fed back into training.

Checklist

- ✓ What it is: live feedback capture + 8-type export
- ✓ Where it lives: `dataset/generator.py`
- ✓ How to trigger it: runs automatically during `engine.py` processing; export via CLI
- ✓ What files it writes: `var/datasets/*.jsonl`, `var/learning.db`
- ✓ What outputs: JSONL files + manifests per type
- ✓ What can fail: SQLite lock, disk space, quality threshold not met
- ✓ How to troubleshoot: check `var/datasets/review_queue.jsonl`

Export types

Type	What it contains
<code>conversation</code>	SFT pairs from user-rated conversations
<code>retrieval</code>	Query + relevant document chunks with quality scores
<code>kg</code>	Text + extracted triple labels
<code>memory</code>	Store/recall pairs with relevance scores
<code>teaching</code>	Level-labelled instructional pairs
<code>emotion</code>	Text + 28-class emotion labels
<code>reasoning</code>	Query + full chain-of-thought reasoning
<code>evaluation</code>	Regression test cases for eval suite

File locations

File	Path
Dataset directory	<code>var/datasets/</code>
Review queue	<code>var/datasets/review_queue.jsonl</code>
Feedback database	<code>var/learning.db</code>
Export files	<code>var/datasets/<type>_<timestamp>.jsonl</code>
Manifests	<code>var/datasets/<type>_manifest.json</code>

How to trigger and review

```
bash
```

```
# Datasets are captured automatically during chat sessions.
# To trigger a manual export:
python main.py generate-corpus var/datasets/

# To review the queue (from the Desktop UI):
# → Dataset tab → Review Queue → Approve / Reject each item

# Approved items can then be registered as a corpus source:
python main.py corpus register-source H-coc-synthetic-v1 \
  --category H \
  --name "COC Synthetic Data v1" \
  --licence "Internal/CC0" \
  --licence-id cc0 \
  --licence-risk 0.0 \
  --raw-path "var/datasets/" \
  --operator mpssp

python main.py corpus run-pipeline H-coc-synthetic-v1 --stages clean,score
# Note: synthetic data always goes to human review queue regardless of score
python main.py corpus review --source H-coc-synthetic-v1 --operator mpssp
```

Reliability limits

- Dataset quality depends entirely on the quality of live sessions
- Low-quality or repetitive conversations produce low-quality datasets
- The quality gate (`min_quality_score = 0.60` in config) filters the worst items
- Always prefer real data sources (Categories A-G) over synthetic (Category H) at ratios > 10:1
- Never increase `auto_approve` to `True` — this bypasses the human review guarantee

5. UI Startup and Usage

Checklist

- ✓ How to start each UI
- ✓ What commands to run, from what directory
- ✓ How auth works, where the token is
- ✓ What panels exist
- ✓ What to do when things fail

5.1 Desktop Application (Primary Interface)

What it is: Native PySide6 application. Calls the Python backend directly — no HTTP layer,

no browser. This is the primary power-user interface.

Requires: `pip install PySide6`

Does not require: PySide6-WebEngine, a browser, the web server

Start

```
bash

# Working directory: cognitiveoc_v3/
python main.py ui --desktop

# Or directly:
python ui/desktop.py
```

What starts: The PySide6 `QMainWindow` launches, initialises the `Engine`, loads all subsystems (memory, retrieval, KG, cognition, guardrails), and opens the main window.

Start time: 5 – 15 seconds (encoder hub loads NPU/CPU models on first run)

Panels

Panel	Tab	What it does
Chat	Left panel	Main conversation interface. Supports streaming (token-by-token).
Memory	 Memory	Browse, search, forget, link memory records.
Knowledge Graph	 KG	Query, explore, export triples.
Workflow	 Workflow	Create, monitor, cancel multi-step workflows.
Guardrails	 Guardrails	Toggle the 12 cognitive guards; switch profiles.
Cognition	 Cognition	Toggle the 9 cognition modules; switch modes.
Metrics	 Metrics	Live hardware (GPU/NPU/CPU/RAM), request latency, subsystem stats.
Evaluation	 Eval	Run the 13-metric evaluation suite.
Dataset	 Dataset	Inspect and review the dataset generation queue.
Training	 Training	Monitor training progress (reads <code>var/checkpoints/train_log.jsonl</code>).

Additional panels in the Advanced window (View menu):

-  Research — multi-loop research sessions
-  Validation — validation engine output inspector
-  Retrieval — retrieval statistics and self-improvement

Auth

The desktop app calls the backend **directly in-process** — no auth token is required. There is no HTTP call between the desktop app and the engine.

Stop

Close the window or press `Ctrl+Q`. The engine saves state to SQLite before exit.

Common failures

Symptom	Cause	Fix
<code>ModuleNotFoundError: PySide6</code>	PySide6 not installed	<code>pip install PySide6</code>
Window opens but panels are blank	Engine init failed	Check console for import errors; check <code>var/</code> exists
Chat sends but no response	Model not loaded	Model weights missing — not yet trained. Chat will fall back to retrieval-only mode
Memory panel empty	No sessions yet	Start chatting; memories are created automatically
Metrics panel shows 0.0	First launch	Observability starts on first request

5.2 Web Application (Secondary Interface)

What it is: Pure-Python HTTP server (`http.server`) — no external framework). Serves a chat frontend at `http://127.0.0.1:8765`. Secondary to the desktop app, suitable for browser-based access or remote sessions.

Does not require: FastAPI, uvicorn, nginx, Node.js

Start

```
bash
```

```
# Working directory: cognitiveoc_v3/  
python main.py ui  
  
# Custom host/port:  
python main.py ui --host 0.0.0.0 --port 9000
```

What starts: HTTP server on `127.0.0.1:8765` (default). Loads the `Engine` on first request. Opens `http://127.0.0.1:8765` in your default browser (if `--no-browser` is not set).

Auth token

How it works:

1. On first startup, a 32-byte random key is generated and saved to `var/auth_key.txt`
2. Every subsequent startup loads the same key from that file
3. The key is displayed at server startup in the terminal
4. All POST endpoints and all GET endpoints (except `/api/auth-key`) require the header: `X-CognitiveOC-Key: <your-key>`
5. The web frontend reads the key automatically from `/api/auth-key` on page load

Where it is stored: `var/auth_key.txt`

How to get it:

```
bash  
  
cat var/auth_key.txt  
# or  
curl http://127.0.0.1:8765/api/auth-key
```

If auth fails: Check that `var/auth_key.txt` exists. If not, the server will generate a new one on next start.

To reset the key: Delete `var/auth_key.txt` and restart the server. All existing sessions will need the new key.

Web API endpoints

Method	Path	What it does
GET	/	Serve <code>ui/static/index.html</code>
GET	/api/auth-key	Return current auth key (no auth required)
GET	/api/status	Full system status JSON
GET	/api/metrics	Observability snapshot
GET	/api/memories	Memory list
GET	/api/kg	KG analytics
GET	/api/retrieval-stats	Retrieval self-improvement stats
GET	/api/workflows	Workflow list
GET	/api/cognition-state	Current cognition mode/modules
GET	/api/guardrail-state	Current guardrail profile/guards
GET	/api/dataset/queue	Dataset review queue
POST	/api/chat	Blocking chat (returns complete response)
POST	/api/stream	SSE streaming chat
POST	/api/upload	Document ingestion
POST	/api/feedback	User rating submission
POST	/api/memory	Memory actions (search, forget, link)
POST	/api/kg	KG actions (query, merge, export)
POST	/api/research	Research engine actions
POST	/api/validate	Validation actions
POST	/api/dataset	Dataset export actions
POST	/api/guardrail-profile	Switch guardrail profile
POST	/api/guardrail-toggle	Toggle one cognitive guard
POST	/api/cognition-mode	Switch cognition mode
POST	/api/cognition-toggle	Toggle one cognition module

Method	Path	What it does
POST	<code>/api/eval</code>	Run evaluation suite

SSE streaming

The web app supports Server-Sent Events (SSE) for streaming chat responses. The frontend automatically uses `/api/stream` when the streaming toggle is on. Heartbeat every 500 ms (`heartbeat_ms` in `config.py`).

Common failures

Symptom	Cause	Fix
<code>Address already in use</code>	Port 8765 occupied	Kill existing process: <code>lsof -ti :8765 xargs kill</code>
Browser shows "Connection refused"	Server not started	Run <code>python main.py ui</code>
Auth error 403	Wrong or missing header	Get key from <code>var/auth_key.txt</code> ; paste into browser
Chat hangs, no response	Model not loaded	Expected before training — system returns a non-model response
SSE stream cuts off	Heartbeat timeout	Check <code>STREAMING.heartbeat_ms</code> in config; default 500 ms

6. Training

What it is

Training happens in two phases:

1. **Tokenizer training** — one-time, fast (minutes to hours)
2. **700M decoder pre-training** — long, segmented, takes months at 16 h/day

Both require the v1 release to be built, signed, and locked first.

Checklist

- ☑ Where training lives: `train/train_model.py` (model), `train/train_tokenizer.py` (tokenizer)
- ☑ What commands to use: `python main.py train-tokenizer` / `python main.py train-model`

- ✓ What files are written: checkpoints in `var/checkpoints/`
- ✓ How to stop and resume safely: Ctrl+C → shard reset → resume-verify → resume
- ✓ What training control files exist: ledger, shard tracker, provenance
- ✓ What can fail: VRAM OOM, checkpoint corruption, release lock mismatch

Pre-training checklist (must complete before starting)

```
□ CORPUS_WAREHOUSE_DIR set to your SSD path in config.py
□ All source families acquired, pipelined, and approved
□ python main.py corpus build-release v1          (release built)
□ python main.py corpus verify-release v1         (checksums pass)
□ python main.py corpus sign-release v1 --operator mpssp
□ python main.py lock-release v1 --operator mpssp
□ python main.py train-tokenizer releases/v1/train.txt (tokenizer trained)
□ python main.py eval --tokenizer                (fertility ≥ 3.5 chars/token)
```

Phase A: Tokenizer training

```
bash

# Working directory: cognitiveoc_v3/
python main.py train-tokenizer releases/v1/train.txt

# Optional overrides:
python main.py train-tokenizer releases/v1/train.txt \
  --prefix coc_tokenizer \
  --vocab-size 48000

# Where releases/ points: D:/corpus_warehouse/releases/v1/train.txt
# Output: var/tokenizer/coc_tokenizer.model + coc_tokenizer.vocab
# Duration: minutes to a few hours depending on corpus size
```

Success looks like:

```
Trained tokenizer: var/tokenizer/coc_tokenizer.model
Vocab size: 48000
Fertility: 3.7 chars/token
```

Failure looks like:

```
FileNotFoundError: releases/v1/train.txt
→ Fix: Run build-release v1 first

ValueError: too few characters for vocab size
→ Fix: Corpus is too small – acquire more sources
```

Phase B: Pre-training the 700M decoder

Session 1 (fresh start)

```
bash

# Step 1: Run pre-flight guard (required before every session)
python main.py resume-verify v1 --fresh --operator mpssp
# Expected: 8 checks, all ✓, "Training may proceed."

# Step 2: Start training
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt

# Optional flags:
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt \
  --steps 100000 \
  --precision bf16
```

Sessions 2+ (resume)

```
bash

# Step 1: Run pre-flight guard (required – verifies release lock integrity)
python main.py resume-verify v1 --operator mpssp
# Expected: all ✓ including "release_locked" and "manifest_hash_tracker"

# Step 2: Resume (auto-loads last checkpoint)
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt --resume

# To force fresh start (discards all progress – destructive):
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt --no-resume
```

What happens during training

1. Resume guard verifies the release lock
2. Shard tracker initialised (fresh) or loaded (resume)
3. Next unconsumed shard is retrieved
4. Training ledger opens a new run entry
5. Training loop runs with live TUI display (loss, perplexity, gradient norm, tokens/s)
6. Checkpoint saved every `save_every=1000` steps to `var/checkpoints/model_700m.pt`
7. Val loss evaluated every `val_every=500` steps
8. Training metrics logged every `log_every=50` steps to `var/checkpoints/train_log.jsonl`
9. When shard completes: marked in shard tracker, next shard fetched

10. On Ctrl+C: current shard reset to `pending`, checkpoint saved, ledger closed with `interrupted` status

Checkpoint files

File	Contents
<code>var/checkpoints/model_700m.pt</code>	Model weights + optimizer + scheduler + step
<code>var/checkpoints/model_700m_best.pt</code>	Best val-loss checkpoint
<code>var/checkpoints/train_log.jsonl</code>	Per-step metrics (step, loss, val_loss, perplexity, lr, grad_norm, tok/s)
<code>var/checkpoints/training_ledger.jsonl</code>	Per-session records (append-only)
<code>var/checkpoints/shard_tracker.json</code>	Shard consumption state
<code>var/checkpoints/provenance.json</code>	Permanent training provenance

Training control commands

```
bash

python main.py training-ledger      # show all training sessions
python main.py shard-status         # show shard consumption progress
python main.py provenance-report    # full model provenance
python main.py resume-verify v1     # run guard without starting training
```

Release locking / unlocking

```
bash

# Lock (do before first training session)
python main.py lock-release v1 --operator mpssp --notes "Phase 1"

# Check lock status
python main.py corpus verify-release v1

# Admin: break lock (only for recalled releases)
python main.py unlock-release v1 --operator mpssp --reason "Recall - bad source"
```

Training time estimates

Tokens/s estimate is for RTX 5060 8GB, bf16, micro-batch=2, accum=16, gradient checkpointing ON.

Phase gates

Run evaluation at 30K, 60K, 100K steps:

```
bash

python main.py eval
python main.py gate 1      # Phase 1 gate (after 30K steps)
```

7. Observability

What it is

The observability system tracks hardware metrics, per-request metrics, subsystem health, and training metrics in real time.

Checklist

- ✓ Where it lives: `observability/metrics.py`
- ✓ How to access: Desktop UI ( Metrics tab) or `GET /api/metrics`
- ✓ What it writes: `var/metrics.json`, `var/logs/runtime.jsonl`,
`var/checkpoints/train_log.jsonl`
- ✓ No separate auth — it's part of the engine

Metrics tracked

Hardware (polled every 2 seconds):

Metric	Meaning
<code>cpu_pct</code>	CPU usage percentage
<code>gpu_vram_used_gb</code>	GPU VRAM used (GB)
<code>gpu_util_pct</code>	GPU utilisation %
<code>gpu_temp_c</code>	GPU temperature (°C)
<code>npu_memory_gb</code>	NPU memory used (AMD via DirectML)
<code>ram_used_gb</code>	System RAM used
<code>ram_available_gb</code>	System RAM available

Per-request:

Metric	Meaning
<code>latency_ms</code>	Total request latency
<code>tokens_in</code>	Input tokens
<code>tokens_out</code>	Output tokens
<code>tokens_per_sec</code>	Generation speed
<code>intent</code>	Classified intent type
<code>backend</code>	<code>model</code> / <code>fallback</code>
<code>retrieval_mode</code>	<code>hybrid</code> / <code>bm25</code> / <code>semantic</code>

Subsystem health:

Subsystem	Metrics
Retrieval	<code>hit_rate</code> , <code>cache_hit_rate</code> , <code>avg_score</code> , <code>multi_hop_rate</code>
Memory	<code>recall_count</code> , <code>store_count</code> , <code>avg_score</code>
KG	<code>query_count</code> , <code>extract_count</code> , <code>triple_count</code>
Cognition	<code>mode</code> , <code>emotion_detected</code> , <code>intent_detected</code>
Reasoning	<code>avg_confidence</code> , <code>type_distribution</code>
Workflow	<code>active_count</code> , <code>completed_count</code> , <code>failed_count</code>

Training (read from `train_log.jsonl`):

Metric	Meaning
<code>step</code>	Current training step
<code>train_loss</code>	Training cross-entropy loss
<code>val_loss</code>	Validation loss
<code>perplexity</code>	<code>exp(val_loss)</code>
<code>grad_norm</code>	Gradient norm (should stay < 1.0 after warmup)
<code>lr</code>	Current learning rate
<code>tokens_per_sec</code>	Training throughput

How to access

bash

```
# Desktop UI: open app → click 📊 Metrics tab

# Web API:
curl -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  http://127.0.0.1:8765/api/metrics

# Direct Python:
from observability.metrics import Observability
obs = Observability()
print(obs.snapshot())

# Training metrics file:
tail -f var/checkpoints/train_log.jsonl
```

Interpreting training metrics

Metric	Good	Concern	Bad
<code>train_loss</code>	Steadily decreasing	Plateau for >5K steps	Increasing
<code>val_loss</code>	Close to <code>train_loss</code>	Rising while <code>train_loss</code> falls	> <code>train_loss</code> + 1.0
<code>perplexity</code>	< 20 after 30K steps	50-100 after 30K	> 200
<code>grad_norm</code>	0.1 - 0.8	1.0 - 2.0	> 5.0 (exploding)
<code>tokens_per_sec</code>	1800 - 2200	1000 - 1800	< 500

If `grad_norm` spikes above 5.0: `grad_clip = 1.0` in config should prevent this. If it happens, reduce `lr` or reduce batch size.

If `val_loss` diverges from `train_loss`: Possible overfitting or data quality issue. Run `python main.py corpus audit-report` to check for recently approved low-quality sources.

8. Memory · Retrieval · KG · Reasoning · Research · Validation

8.1 Memory System

- File: `memory/memory.py`
- Database: `var/memory.db` (SQLite with FTS5)
- Config: `MEMORY` block in `config.py`

8 memory types

Type	What it stores
Episodic	Individual interaction events with timestamps
Semantic	Generalised facts about the world / domain
Preference	User preferences and communication style
Project	Project-specific context and decisions
Task	Task state, progress, steps
Goal	Long-term goals (synced with GoalTracker)
Learning	Topics the user has studied; knowledge state
Workspace	Active document/workspace context

Ranked recall (4-factor)

```
score = relevance_weight × relevance
      + recency_weight   × recency
      + frequency_weight × frequency
      + importance_weight× importance
```

```
Default weights (config.py MEMORY):
relevance_weight  = 0.10
recency_weight    = 0.30
frequency_weight  = 0.20
importance_weight = 0.40
```

Access

```
bash
```



```
# Desktop UI: 🧠 Memory tab
#   - Search box: full-text search across all memory types
#   - Forget button: remove a memory record
#   - Link button: create association between memories
#   - Filter by type: dropdown selector

# Web API:
curl -X POST -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  -H "Content-Type: application/json" \
  -d '{"action": "search", "query": "my project"}' \
  http://127.0.0.1:8765/api/memory

# CLI:
python main.py chat    # memories are stored/recalled automatically
```

Troubleshoot

Issue	Cause	Fix
Memory panel blank	No sessions yet	Start chatting
Memory not recalled	Relevance too low	Rephrase — similarity must exceed threshold
<code>var/memory.db</code> corrupted	Power loss during write	Delete and restart — memories will rebuild from logs

8.2 Retrieval System

File: `retrieval/rag.py`

Index: `var/index/` (embedding vectors + metadata)

Cache: `var/cache/` (retrieval cache + analytics)

Workspaces: `var/workspaces/`

Architecture

```
HybridRetriever.retrieve(query)
  → query rewrite (history-aware)
  → query expansion (synonyms + encoder)
  → RAGPipeline.retrieve()    BM25 + semantic cosine hybrid
  → CAGManager.retrieve()     Active document cache
  → CrossEncoder.rerank()     Precision reranking (BGE cross-encoder)
  → CitationEngine.track()    Chunk provenance
  → multi-hop (up to 3 hops if low confidence)
```

Ingest documents

```
bash

# Via CLI:
python main.py ingest /path/to/document.pdf

# Via Desktop UI: File → Ingest Document (or drag-drop onto Chat)

# Via Web API:
curl -X POST -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  -F "file=@document.pdf" \
  http://127.0.0.1:8765/api/upload
```

Inspect retrieval stats

```
bash

# Desktop UI: Advanced → 🔍 Retrieval tab

# Web API:
curl -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  http://127.0.0.1:8765/api/retrieval-stats
```

Troubleshoot

Issue	Cause	Fix
Retrieval returns nothing	No documents indexed	Ingest documents first
Poor retrieval quality	BGE model not loaded	Check NPU/CPU encoder initialisation
<code>var/index/</code> missing	First run	It is created automatically on first ingest

8.3 Knowledge Graph

File: `knowledge/graph.py`
Database: `var/knowledge_graph.db` (SQLite + FTS5)
Capacity: 500 K triples · 100 K entities

What it stores

Triples in the form `(subject, relation, object, confidence, source)`. Automatically populated by the KG encoder during chat sessions.

Access

```
bash

# Desktop UI: 🏠 KG tab
#   - Query box: natural language → SPARQL-like lookup
#   - Entity search: find all triples for an entity
#   - Export: dump triples to JSON

# Web API:
curl -X POST -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  -H "Content-Type: application/json" \
  -d '{"action": "query", "entity": "Python"}' \
  http://127.0.0.1:8765/api/kg

# Ingest documents to KG:
python main.py ingest /path/to/doc.pdf # same command – KG extraction is auto
```

8.4 Reasoning Engine

File: `reasoning/reasoner.py`

The reasoning engine activates automatically for complex queries (detected by intent classifier). It produces a step-by-step chain of thought that is injected into the decoder context.

```
bash

# To see reasoning traces: Desktop UI → Chat panel → expand [Reasoning] block
# Or inspect the <reasoning>...</reasoning> tags in raw responses via API
```

8.5 Research Engine

File: `research/engine.py`

Config: `RESEARCH` block — max 10 loops, min 3 evidence pieces, 2 validation rounds

The research engine runs multi-loop synthesis: it retrieves evidence, synthesises a draft, validates it, and iterates up to `max_loops` times.

```
bash
```

```
# Desktop UI: Advanced → 📄 Research tab
#   - Enter a research question
#   - Set loop count and evidence minimum
#   - Start research
#   - Output is a markdown report with citations

# Web API:
curl -X POST -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  -H "Content-Type: application/json" \
  -d '{"action": "start", "query": "Explain transformer attention mechanisms", "loop": 10}' \
  http://127.0.0.1:8765/api/research
```

8.6 Validation Engine

File: validation/validator.py

Config: VALIDATION block — fact check, citation check, KG check, memory check, reasoning check all enabled by default

Validation runs automatically after every response. It checks:

Check	What it does
Fact check	Compares claims against retrieved context
Citation check	Verifies cited sources exist in index
KG check	Cross-references claims against KG triples
Memory check	Verifies memory-based claims are consistent
Reasoning check	Validates logical consistency of reasoning trace

```
bash

# Desktop UI: Advanced → ✅ Validation tab – shows validation result per response

# Web API:
curl -X POST -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  -H "Content-Type: application/json" \
  -d '{"action": "validate", "text": "Python was created in 1991 by Guido van Rossum"}' \
  http://127.0.0.1:8765/api/validate
```

9. CLI Reference

Working directory for all commands: `cognitiveoc_v3/`

Core commands

Command	What it does	Key files
<code>python main.py chat</code>	Interactive CLI chat session	<code>engine.py</code>
<code>python main.py ui</code>	Start web server on port 8765	<code>ui/app.py</code>
<code>python main.py ui --desktop</code>	Launch desktop app	<code>ui/desktop.py</code>
<code>python main.py status</code>	Print full system health	<code>engine.py</code> , <code>config.py</code>
<code>python main.py eval</code>	Run 13-metric eval suite	<code>eval/run_suite.py</code>
<code>python main.py gate <N></code>	Run phase N gate check	<code>eval/run_suite.py</code>

Corpus commands

Command	What it does
<code>python main.py corpus register-source <id> --category <A-K> ...</code>	Register new source
<code>python main.py corpus validate-source <id> --operator <you></code>	Validate / approve source
<code>python main.py corpus validate-source <id> --reject --reason "..."</code>	Reject source
<code>python main.py corpus run-pipeline <id> --stages clean,dedup,score</code>	Run pipeline stages
<code>python main.py corpus review --source <id> --operator <you></code>	Interactive review
<code>python main.py corpus review --stats</code>	Show queue counts
<code>python main.py corpus build-release <ver> --categories ... --token-budget ...</code>	Build release
<code>python main.py corpus build-release <ver> --dry-run</code>	Dry-run release build
<code>python main.py corpus verify-release <ver></code>	Verify checksums + leakage

Command	What it does
<code>python main.py corpus sign-release <ver> --operator <you></code>	Sign release
<code>python main.py corpus list-releases</code>	List all releases
<code>python main.py corpus warehouse-stats</code>	Warehouse token + storage stats
<code>python main.py corpus audit-report</code>	Full audit report
<code>python main.py corpus audit-report --source <id></code>	Per-source compliance
<code>python main.py corpus training-ledger</code>	Training session ledger
<code>python main.py corpus shard-status</code>	Shard tracker state
<code>python main.py corpus resume-verify <ver> [--fresh]</code>	Pre-training guard
<code>python main.py corpus provenance-report</code>	Training provenance
<code>python main.py corpus lock-release <ver> --operator <you></code>	Lock release
<code>python main.py corpus unlock-release <ver> --operator <you> --reason "..."</code>	Break lock (admin)

Training commands

Command	What it does
<code>python main.py train-tokenizer <corpus></code>	Train 48K SentencePiece tokenizer
<code>python main.py train-model <corpus></code>	Train 700M decoder (fresh start)
<code>python main.py train-model <corpus> --resume</code>	Resume from last checkpoint
<code>python main.py train-model <corpus> --no-resume</code>	Force fresh start
<code>python main.py training-ledger</code>	Print training ledger
<code>python main.py shard-status</code>	Print shard tracker
<code>python main.py resume-verify <ver> [--fresh] [--operator <you>]</code>	Run pre-training guard
<code>python main.py provenance-report</code>	Training provenance

Command	What it does
<code>python main.py lock-release <ver> --operator <you> --notes "..."</code>	Lock a release
<code>python main.py unlock-release <ver> --operator <you> -reason "..."</code>	Break lock (admin)

Dataset and data commands

Command	What it does
<code>python main.py generate-corpus <dir></code>	Export pending dataset items
<code>python main.py prepare-corpus <path></code>	Run base data pipeline (clean/split/manifest)
<code>python main.py validate-corpus <path></code>	Validate a corpus file
<code>python main.py ingest <path></code>	Ingest document into retrieval index and KG

10. File and Folder Reference

Repository root (`cognitiveoc_v3/`)

Path	Contents	Edit?
<code>config.py</code>	All configuration — single source of truth	Add keys only
<code>engine.py</code>	Orchestration core, intent router	No
<code>main.py</code>	CLI entry point — all subcommands	Add commands only

New subsystem packages

Package	Files	Purpose
<code>corpus/</code>	<code>cleaner</code> , <code>scorer</code> , <code>dedup</code> , <code>reviewer</code> , <code>manifest</code> , <code>release_builder</code> , <code>warehouse</code> , <code>source_registry</code> , <code>cli</code>	12-stage corpus pipeline
<code>governance/</code>	<code>source_registry.json</code> , <code>license_rules.json</code> , <code>approval_log.jsonl</code> , <code>audit.py</code>	Source governance data
<code>audit/</code>	<code>logger.py</code> , <code>reporter.py</code>	Append-only audit logging

Package	Files	Purpose
<code>release/</code>	lock.py, verify.py, v1/manifest.json	Release management
<code>train/</code> (additions)	training_ledger.py, shard_tracker.py, resume_guard.py, provenance.py	Training control

Runtime data (`var/` — not in repository)

Path	Contents	Notes
<code>var/memory.db</code>	8-type memory database	SQLite — do not manually edit
<code>var/knowledge_graph.db</code>	Knowledge graph triples	SQLite — do not manually edit
<code>var/index/</code>	Retrieval embedding vectors	Rebuilt by ingesting documents
<code>var/cache/</code>	Retrieval cache + analytics	Safe to delete (rebuilt)
<code>var/workspaces/</code>	Active workspace sessions	Persists between restarts
<code>var/datasets/</code>	Generated dataset JSONL files	Review before using
<code>var/learning.db</code>	Feedback SQLite database	Feeds dataset generator
<code>var/tokenizer/</code>	Trained tokenizer model + vocab	Critical — do not delete after training
<code>var/checkpoints/</code>	Model checkpoints + training control	Critical — back up
<code>var/logs/</code>	Runtime JSONL logs	Safe to delete
<code>var/metrics.json</code>	Latest observability snapshot	Regenerated each session
<code>var/auth_key.txt</code>	Web API auth key	Keep private
<code>var/cognition_state.json</code>	Cognition module on/off state	Safe to reset
<code>var/guardrails_state.json</code>	Guardrail profile + guard state	Safe to reset
<code>var/onnx/</code>	NPU ONNX encoder models	Regenerated if missing
<code>var/uploads/</code>	Uploaded documents	Safe to clear

Critical file locations

11. Startup and Shutdown

Full system startup

```
bash

# 1. Navigate to repo root
cd /path/to/cognitiveoc_v3

# 2. Start desktop app (primary – starts the full backend)
python main.py ui --desktop

# OR start web server (secondary)
python main.py ui

# Backend auto-starts on first request.
# On startup, ensure_dirs() creates all var/ subdirectories if missing.
```

Startup sequence (what happens internally)

1. `config.py` loaded — paths resolved, `ensure_dirs()` creates `var/` tree
2. `Engine.__init__()` called — loads all subsystems
3. Auth key loaded from `var/auth_key.txt` (or generated fresh)
4. Encoder hub initialises 13 encoders (NPU preferred, CPU fallback)
5. Memory system opens `var/memory.db`
6. Retrieval system opens `var/index/`
7. KG opens `var/knowledge_graph.db`
8. Cognition layer loads `var/cognition_state.json`
9. Guardrails load `var/guardrails_state.json`
10. Observability starts hardware polling thread

Shutdown

```
bash

# Desktop app: close window or Ctrl+Q
# Web server: Ctrl+C in terminal

# Both: SQLite databases are closed cleanly on exit.
# If killed abruptly (power loss): databases may have unwritten WAL pages.
# SQLite WAL mode handles this – restart will recover automatically.
```

Recovery after crash or power loss

```
bash

# 1. Check database integrity
python -c "import sqlite3; c=sqlite3.connect('var/memory.db'); print('OK')"
python -c "import sqlite3; c=sqlite3.connect('var/knowledge_graph.db'); print('OK')"

# 2. If corrupted: delete and restart (data lost)
rm var/memory.db var/knowledge_graph.db
python main.py status # reinitialises on first run

# 3. If training checkpoint is suspect:
python main.py resume-verify v1 --operator mpssp
# The guard will report checkpoint integrity

# 4. If shard tracker has stuck shards:
python main.py shard-status
# Stuck in_progress shards are auto-reset by resume-verify
```

12. Troubleshooting

System startup failures

Symptom	Cause	How to check	Fix
<code>ModuleNotFoundError: PySide6</code>	Not installed	<code>python -c "import PySide6"</code>	<code>pip install PySide6</code>
<code>ModuleNotFoundError: torch</code>	PyTorch not installed	<code>python -c "import torch"</code>	<code>pip install torch</code>
<code>SyntaxError</code> in any module	Python < 3.10	<code>python --version</code>	Use Python 3.10+
<code>ensure_dirs()</code> fails	Disk full or permission denied	<code>df -h</code>	Free disk space
Port 8765 in use	Another process	<code>lsof -i :8765</code>	Kill other process or use <code>--port 9000</code>

Auth failures

Symptom	Cause	How to check	Fix
403 on all POST requests	Wrong auth key	<code>cat var/auth_key.txt</code>	Paste correct key into browser header
Auth key changed between runs	Key file deleted	<code>ls -la var/auth_key.txt</code>	Key is regenerated on first start; get new key
<code>/api/auth-key</code> returns empty	Server not started	<code>curl http://127.0.0.1:8765/api/auth-key</code>	Start server first

UI failures

Symptom	Cause	Fix
Desktop panel blank	Backend init error	Check terminal for Python traceback

Symptom	Cause	Fix
Chat responds but no memory	Memory DB not writable	Check <code>(var/)</code> permissions
Stream hangs after first token	SSE heartbeat timeout	Check <code>(STREAMING.heartbeat_ms)</code> in config (default 500)
Web page loads but chat returns 500	Engine error	Check terminal output for exception

Corpus failures

Symptom	Cause	Fix
<code>(release directory not found)</code> on verify	Release not built	<code>(python main.py corpus build-release v1)</code>
Checksum mismatch on verify	File modified after build	Rebuild release — never manually edit split files
<code>(Source not found in registry)</code>	Not registered	<code>(python main.py corpus register-source ...)</code>
Pipeline fails with path error	Raw path does not exist	Create the directory and copy files before running
Review queue grows but never cleared	Borderline quality	Raise <code>(auto_approve_threshold)</code> or use better source
<code>(release not locked)</code> on resume-verify	Lock not applied	<code>(python main.py lock-release v1 --operator mpssp)</code>
Lock mismatch on resume-verify	Release files modified after locking	This is a data integrity violation — investigate before proceeding

Training failures

Symptom	Cause	Fix
<code>(CUDA out of memory)</code>	VRAM too low	<code>(micro-batch=1)</code> in config, keep gradient checkpointing ON
<code>(KeyError: 'step' in checkpoint)</code>	Checkpoint malformed	Delete checkpoint, restart fresh

Symptom	Cause	Fix
Training loss NaN after resume	Optimizer state mismatch	Delete checkpoint, restart fresh from last known good
<code>ResumeGuardError: release not locked</code>	Lock missing	<code>python main.py lock-release v1 --operator mpssp</code>
<code>ResumeGuardError: hash mismatch</code>	Release file changed after locking	Investigate who changed files; rebuild if necessary
Shard stuck in <code>in_progress</code>	Previous session killed	<code>python main.py resume-verify v1</code> auto-resets stuck shards
<code>train_loss</code> not decreasing	Learning rate too high	Lower <code>lr</code> in config from <code>3e-4</code> to <code>1e-4</code>
<code>grad_norm</code> exploding (>10)	LR too high or bad batch	Lower LR; check corpus for corrupted paragraphs

Dataset generation failures

Symptom	Cause	Fix
<code>var/datasets/</code> empty	No sessions captured yet	Run chat sessions; datasets accumulate automatically
Review queue never populates	<code>min_quality_score</code> too high	Lower threshold in <code>DATASET</code> config (default 0.60)
Dataset export fails	SQLite lock on <code>var/learning.db</code>	Only one process should write at a time

13. End-to-End Flow

This section traces the complete path from a fresh clone to a trained, running system.

Step 1: Initial setup

```
bash
```

```
# Clone / open the repository
cd cognitiveoc_v3/

# Verify Python
python --version    # must be 3.10+

# Install dependencies
pip install torch --index-url https://download.pytorch.org/whl/cu124
pip install PySide6 sentencepiece transformers accelerate

# Verify all modules load
python -c "import config, engine, corpus.cli, train.training_ledger; print('OK'"

# Verify CLI
python main.py --help
```

Step 2: Set configuration

Open `config.py` and set:

```
python

CORPUS_WAREHOUSE_DIR = Path("D:/corpus_warehouse")    # ← YOUR 1TB SSD PATH
```

Everything else can stay at defaults for the first run.

Step 3: Start the system (before corpus work)

```
bash

# Start desktop app – confirms everything loads correctly
python main.py ui --desktop
# Expected: window opens, status bar shows "Backend: Ready"
# Chat will work in fallback mode (no model yet)

# Or start web server:
python main.py ui
# Expected: "CognitiveOC server listening on http://127.0.0.1:8765"
# Browser opens automatically
```

Step 4: Verify infrastructure

```
bash
```

```
python main.py corpus warehouse-stats
# Expected: all categories at 0 tokens, no errors

python main.py corpus audit-report
# Expected: "Total events: 0"

python main.py status
# Expected: all subsystems initialised
```

Step 5: Register and process corpus sources

Repeat this block for each of the 11 source categories (A through K):

```
bash

# Example: Category A (Project Gutenberg)

# 5a. Register the source
python main.py corpus register-source A-gutenberg-20260701 \
  --category A \
  --name "Project Gutenberg English" \
  --url https://gutenberg.org \
  --licence "Public Domain" --licence-id pd --licence-risk 0.0 \
  --raw-path "D:/corpus_warehouse/raw/books/gutenberg/" \
  --operator mpssp

# 5b. Download raw files
#   → place .txt files in D:/corpus_warehouse/raw/books/gutenberg/
#   → use Project Gutenberg bulk download or mirror

# 5c. Validate (confirms licence + content review done)
python main.py corpus validate-source A-gutenberg-20260701 --operator mpssp

# 5d. Run pipeline
python main.py corpus run-pipeline A-gutenberg-20260701 \
  --stages clean,dedup,score --operator mpssp --verbose

# 5e. Review borderline items (if any)
python main.py corpus review --source A-gutenberg-20260701 --operator mpssp

# 5f. Check progress
python main.py corpus warehouse-stats
```

Step 6: Generate COC synthetic data (Category H)

```
bash
```

```
# 6a. Run chat sessions to capture feedback
python main.py chat          # or use the desktop/web UI
# Use the system for several hours – datasets accumulate automatically

# 6b. Register synthetic output as a source
python main.py corpus register-source H-coc-synthetic-v1 \
  --category H \
  --name "COC Synthetic Data v1" \
  --licence "Internal/CC0" --licence-id cc0 --licence-risk 0.0 \
  --raw-path "var/datasets/" \
  --operator mpssp

python main.py corpus validate-source H-coc-synthetic-v1 --operator mpssp
python main.py corpus run-pipeline H-coc-synthetic-v1 --stages clean,score

# 6c. Review ALL synthetic items (mandatory – no auto-approve)
python main.py corpus review --source H-coc-synthetic-v1 --operator mpssp
```

Step 7: Build and lock the v1 release

```
bash

# 7a. Dry run – check what would be built
python main.py corpus build-release v1 \
  --categories A,B,C,D,E,F,G,H,I,J,K --dry-run

# 7b. Build for real
python main.py corpus build-release v1 \
  --categories A,B,C,D,E,F,G,H,I,J,K

# 7c. Verify checksums and leakage detection
python main.py corpus verify-release v1
# Expected: all checks ✓

# 7d. Sign (human approval)
python main.py corpus sign-release v1 --operator mpssp

# 7e. Lock (immutable for training)
python main.py lock-release v1 --operator mpssp \
  --notes "v1 30B token release – Phase 1 pre-training"

# 7f. Confirm
python main.py corpus list-releases
# Expected: v1  status=signed  locked=YES
```


Step 8: Train the tokenizer

```
bash

# Working directory: cognitiveoc_v3/
python main.py train-tokenizer D:/corpus_warehouse/releases/v1/train.txt

# Output: var/tokenizer/coc_tokenizer.model + coc_tokenizer.vocab
# Duration: minutes to hours

# Verify fertility
python main.py eval --tokenizer
# Expected: fertility ≥ 3.5 chars/token
```

Step 9: First training session

```
bash

# Pre-flight check (always run before first or resumed session)
python main.py resume-verify v1 --fresh --operator mpssp
# Expected: 8 checks all ✓

# Start training
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt
# Expected: TUI shows step, loss, perplexity, grad_norm, tokens/s
#           Checkpoint saved every 1000 steps to var/checkpoints/model_700m.pt
```

Step 10: Stop and resume training

```
bash

# To stop: press Ctrl+C
# Expected: "Saving checkpoint before exit..."
#           Shard reset to pending
#           Ledger entry closed with status=interrupted

# --- next day ---

# Pre-flight check
python main.py resume-verify v1 --operator mpssp
# Expected: all ✓ including checkpoint integrity

# Resume
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt --resume
# Expected: "Resuming from step N..."
```

Step 11: Monitor during training

```
bash

# In a separate terminal:
tail -f var/checkpoints/train_log.jsonl

# Check ledger
python main.py training-ledger

# Check shard progress
python main.py shard-status

# Full observability
curl -H "X-CognitiveOC-Key: $(cat var/auth_key.txt)" \
  http://127.0.0.1:8765/api/metrics
```

Step 12: Evaluate

```
bash

# Run at 30K, 60K, 100K steps:
python main.py eval

# Phase gates:
python main.py gate 1      # after 30K steps
python main.py gate 2      # after 60K steps
python main.py gate 3      # after 100K steps

# Provenance
python main.py provenance-report
```

14. Frozen vs Planned

Frozen baseline — do not modify

Module	Why frozen
models/transformer.py	700M architecture; modifying breaks all existing checkpoints
cognition/cognition.py	Isolation guarantee must be preserved
safety/guardrails.py	Hard integrity guards must always be on
memory/memory.py	SQLite schema; changes corrupt stored data

Module	Why frozen
<code>retrieval/rag.py</code>	Index format; changes break existing indexes
<code>knowledge/graph.py</code>	Triple schema; changes corrupt the KG
<code>inference/generator.py</code>	Must match decoder exactly
<code>eval/run_suite.py</code>	Benchmark comparability
<code>train/train_model.py</code>	Extend only; never rewrite
<code>data/pipeline.py</code>	Base pipeline; corpus modules depend on it
<code>tokenizer/tokenizer.py</code>	Any change requires retraining from scratch
<code>ui/app.py</code>	Web API contract; frontend depends on endpoints
<code>ui/desktop.py</code>	Qt layout; modifying breaks existing panel bindings

Currently implemented and working

Item	Status
700M decoder (architecture)	✓ Frozen, verified
48K tokenizer	✓ Architecture frozen; training not yet done
Encoder hub (13 encoders)	✓ Frozen, verified
Human cognition layer	✓ Frozen, 9 modules
Memory system	✓ Frozen, 8 types
Retrieval system	✓ Frozen, hybrid BM25+semantic
Knowledge graph	✓ Frozen, 500K triples
Reasoning engine	✓ Frozen
Research engine	✓ Frozen
Validation engine	✓ Frozen
Workflow engine	✓ Frozen
Guardrail framework	✓ Frozen
Dataset generator	✓ Frozen, 8 types

Item	Status
Observability	✓ Frozen
Web UI (18 endpoints)	✓ Frozen
Desktop UI (13 panels)	✓ Frozen
Corpus pipeline (12 stages)	✓ Implemented, CLI wired
Source registry + governance	✓ Implemented
Audit system	✓ Implemented
Release builder + locking	✓ Implemented
Training ledger	✓ Implemented
Shard tracker	✓ Implemented
Resume guard	✓ Implemented
Training provenance	✓ Implemented
All CLI commands	✓ 17 top-level commands + 16 corpus subcommands

Not yet done (operational tasks, not code gaps)

Item	What is needed
Tokenizer training	Run <code>python main.py train-tokenizer</code> on v1 release
Corpus source acquisition	Download and pipeline all 11 source families
v1 release build	Build after all sources acquired
700M decoder training	Run after tokenizer + release are ready
Corpus panel in desktop UI	Minor extension to <code>ui/desktop.py</code> — not yet added
SFT fine-tuning	After pre-training completes
ONNX export for NPU inference	After pre-training completes

Config keys to set before starting

python

```
# config.py – required before any corpus or training work:
CORPUS_WAREHOUSE_DIR = Path("D:/corpus_warehouse") # ← set to your SSD path

# These are already set correctly and should not change:
TRAIN.steps           = 100_000
TRAIN.batch           = 2
TRAIN.accum_steps     = 16
CORPUS.target_v1_tokens = 30_000_000_000
CORPUS.split_seed     = 42
```

CognitiveOC v3 — Complete User and Developer Guide
79 Python modules · 115 total files · All syntax-clean · All CLI commands verified and running

A. Corpus Engineering Handbook

What this section is: A complete reference for the governed corpus subsystem — why it exists, how it protects training quality, and how every stage works.

A.1 What the Corpus System Is

The corpus system is a **12-stage governed data pipeline**, not a folder of text files. It exists to answer three questions with certainty at any point in time:

- 1. **What data trained this model?** — answered by the release manifest and provenance record
- 2. **Is that data legally safe?** — answered by the licence registry and zero-risk policy enforcement
- 3. **Is that data high quality?** — answered by the quality gate, deduplication, and review queue

Without a governed corpus system, training reproducibility is impossible and licence liability is unknown. COC enforces governance at every stage, with no silent skipping.

A.2 Why It Exists

Risk without corpus governance	How COC mitigates it
Unknown licence in training data	<code>governance/license_rules.json</code> + per-source risk score
Duplicate content skewing training	MinHash LSH cross-source deduplication

Risk without corpus governance	How COC mitigates it
Low-quality paragraphs degrading model	3-axis quality gate (auto-reject < 0.45)
Synthetic data poisoning	Mandatory 100% human review for Category H
Training on modified data after locking	LOCK file + resume guard hash check
No way to audit what trained the model	Append-only audit log + provenance record
Data contaminating evaluation	Category L holdout, leakage check at release build

A.3 Warehouse vs Release — the Core Distinction

CORPUS WAREHOUSE (1TB SSD)	TRAINING RELEASE (subset)
All approved source families	→ Curated quality-checked snapshot
Long-term storage	Versioned: v1, v2, v3 ...
Append-only	LOCKED after signing
100B+ tokens eventual target	30B tokens (v1 target)
Includes warehouse-only sources	Zero-risk sources only (risk ≤ 0.20)
Source-level manifests	Release manifest + checksums
Never directly trained on	Only thing fed to train_model.py

These must never be merged. The warehouse is the asset; the release is the snapshot.

A.4 The 12-Stage Pipeline

Stage 1	ACQUIRE	→ warehouse/raw/<category>/
Stage 2	VALIDATE	→ licence check + content review → registry update
Stage 3	NORMALIZE	→ UTF-8 paragraphs → warehouse/cleaned/
Stage 4	CLEAN	→ boilerplate/PII/noise removal
Stage 5	DEDUPLICATE	→ within-source (cosine 0.92) + cross-source (MinHash 0.85)
Stage 6	SCORE	→ quality + category + risk per paragraph
Stage 7	REVIEW	→ auto-approve ≥0.70 + ≤0.20 risk; else queue
Stage 8	APPROVE	→ source locked in registry as approved
Stage 9	SPLIT	→ 90% train / 5% val / 5% test, seed=42
Stage 10	RELEASE	→ train.txt + val.txt + test.txt + manifest + checksums
Stage 11	AUDIT	→ every event → governance_logs/audit_YYYYMMDD.jsonl
Stage 12	ARCHIVE	→ raw files → warehouse/archive/ after release signed

No stage may be skipped silently. Every stage writes an audit event before completing. Failure at any stage blocks the next stage.

A.5 Pipeline Stage Details

Stage 1 — Acquire

Purpose: Obtain raw source materials from approved source families only.

Inputs: Source URL, download path, operator name

Outputs: Files in `warehouse/raw/<category>/`

Who runs it: Human operator (manual download) + acquisition helper

On failure: Source stays `pending` in registry; no pipeline runs

Audit event: `acquire → source_acquired`

```
bash
```

```
# After downloading files to the raw path:
```

```
python main.py corpus register-source A-gutenberg-v1 \  
  --category A --licence "Public Domain" --licence-id pd \  
  --licence-risk 0.0 --raw-path "D:/corpus_warehouse/raw/books/gutenberg/" \  
  --operator mpssp
```

Stage 2 — Validate

Purpose: Confirm licence and content suitability before any processing.

Inputs: Source registry entry

Outputs: `source_validated=True` in registry; audit event

Who runs it: Human operator after reviewing the source

On failure: Source status set to `rejected`; pipeline blocked

Audit event: `validate → source_validated` or `source_rejected`

```
bash
```

```
python main.py corpus validate-source A-gutenberg-v1 --operator mpssp
```

```
# To reject:
```

```
python main.py corpus validate-source A-gutenberg-v1 \  
  --reject --reason "OCR quality too low" --operator mpssp
```

Stage 3 — Normalize

Purpose: Convert all formats to UTF-8 plain-text paragraphs.

Inputs: Files in `warehouse/raw/<category>/`

Outputs: UTF-8 `.txt` files in `warehouse/cleaned/<category>/`

Who runs it: Automated (`corpus/cleaner.py`)

Supported formats: `.txt`, `.pdf` (pdfminer), `.epub` (ebooklib), `.html` (html2text), `.jsonl`

Audit event: `normalize → normalize_complete`

Stage 4 — Clean

Purpose: Remove boilerplate, PII, noise from normalised text.

Operations (in order):

1. PG boilerplate strip (header/footer regex)
2. HTML/XML tag removal
3. Running header/footer collapse
4. Figure/table caption removal
5. Reference list removal (papers)
6. Equation replacement (`[equation]` for LaTeX)
7. URL removal
8. PII scan + redaction (email, SSN, phone, CC, API key patterns)
9. Min word count filter (< 8 words → discard)
10. Degenerate content removal

Category-specific cleaners (`corpus/cleaner.py`):

- `clean_gutenberg()` — PG header/footer strip
- `clean_openstax()` — exercise block separation
- `clean_wikipedia()` — template removal, stub detection
- `clean_arxiv_full()` — references + equations strip
- `clean_dolly()`, `clean_oasst2()`, `clean_flan()` — instruction format
- `clean_msmarco()`, `clean_nq()` — retrieval format
- `clean_dbpedia()`, `clean_conceptnet()` — KG format
- `clean_generic()` — fallback

```
bash
```

```
python main.py corpus run-pipeline A-gutenberg-v1 --stages clean --verbose
```

Stage 5 — Deduplicate

Level 1 — Within-source (`data/pipeline.py::dedup()`):

- Cosine similarity threshold: 0.92
- Removes OCR duplicates, same passage in different chapters

Level 2 — Cross-source (`corpus/dedup.py::CrossSourceDeduper`):

- MinHash LSH, 128 hash functions, 5-gram character shingles
- Jaccard threshold: 0.85
- Removes same OpenStax passage appearing in Wikipedia + CK-12 + mirror

- First-registered source wins on collision

Level 3 — Release leakage (release/verify.py):

- Exact SHA-256 paragraph hash between train/val/test
- Zero tolerance — any leakage blocks the release build

```
bash
```

```
python main.py corpus run-pipeline A-gutenberg-v1 --stages dedup --verbose
```

Stage 6 — Score

Purpose: Assign 3-axis quality score to every paragraph.

Quality score (0.0–1.0, 6 components):

```
quality_score = word_count_score      × 0.20    # 20–500 words → 1.0
                + lexical_richness     × 0.20    # type-token ratio + 0.5
                + sentence_variety     × 0.15    # avg 10–30 words/sentence
                + printable_ratio      × 0.10    # printable chars / total
                + pii_clean            × 0.10    # 1.0 if no PII detected
                + non_repetitive       × 0.15    # top-3 words < 30% of total
                + information_density × 0.10    # domain keyword density
```

Category score (0.0–1.0): vocabulary density signals per category (reasoning, technical, instruction, emotion, retrieval, KG).

Risk score (0.0–1.0): looked up from source registry (licence_risk) field.

```
bash
```

```
python main.py corpus run-pipeline A-gutenberg-v1 --stages score --verbose
```

Stage 7 — Review

Condition	Decision
quality ≥ 0.70 AND risk ≤ 0.20	Auto-approve — written to (warehouse/approved/)
quality 0.45–0.70 OR risk 0.20–0.50	Human review queue → (warehouse/review_queue/pending.jsonl)
quality < 0.45 OR risk > 0.50	Auto-reject → (warehouse/rejected/)
Synthetic (any score)	Always human review
Synthetic quality < 0.70	Auto-reject

```
bash
```

```
python main.py corpus review --source A-gutenberg-v1 --operator mpssp  
# Controls: [a]pprove [r]eject [s]kip [q]uit  
python main.py corpus review --stats # queue counts without entering review
```

Stage 8 — Approve

Purpose: Lock source as approved; update registry with paragraph and token counts.

Triggered: After review queue is cleared for a source

Outputs: Source `(status = "approved")` in registry; `(approved_paragraphs)`,
`(approved_tokens_est)` set

Audit event: `(approve → source_approved)`

Stage 9 — Split

Purpose: Produce reproducible train/val/test splits.

Ratios: 90% / 5% / 5%

Seed: 42 (fixed forever)

Stratification: Val and test contain proportional samples from every category

Leakage check: Exact-match between train and val/test before split is accepted

Stage 10 — Release

Purpose: Assemble the final release artifact.

Outputs: `(releases/v1/train.txt)`, `(val.txt)`, `(test.txt)`, `(manifest.json)`, `(checksums.sha256)`

```
bash
```

```
python main.py corpus build-release v1 \  
    --categories A,B,C,D,E,F,G,H,I,J,K --token-budget 300000000000  
python main.py corpus verify-release v1  
python main.py corpus sign-release v1 --operator mpssp
```

Stage 11 — Audit

Purpose: Permanent record of every pipeline decision.

Files written:

- `(governance/approval_log.jsonl)` — in-repo, committed, approval decisions
- `(D:/corpus_warehouse/governance_logs/audit_YYYYMMDD.jsonl)` — daily shards on SSD

Event types: `(acquire)` · `(validate)` · `(normalize)` · `(clean)` · `(dedup)` · `(score)` · `(review)` · `(approve)` ·
`(release)` · `(resume_guard)` · `(training)`

Stage 12 — Archive

Purpose: Long-term preservation after release is complete.

Actions: Raw files → `warehouse/archive/`; recalled releases →

`warehouse/archive/recalled/`

B. Warehouse Operations Manual

B.1 What the Warehouse Is

The corpus warehouse is the **long-term storage layer** for all approved source material. It lives entirely on the 1TB SSD and is never committed to the git repository. The repository stores only metadata (registry JSON, approval log, code).

B.2 Setting the Warehouse Path

File: `config.py`

Key: `CORPUS_WAREHOUSE_DIR`

```
python
```

```
# config.py – MUST be set before any corpus work
CORPUS_WAREHOUSE_DIR = Path("D:/corpus_warehouse") # change to your SSD path
```

Verify it is set correctly:

```
bash
```

```
python main.py corpus warehouse-stats
# Expected: shows all categories at 0 tokens, no errors
```

B.3 Complete Folder Layout

```
D:/corpus_warehouse/
|
├─ raw/                                STAGE 1 – immutable source archives
|   │
|   ├─ books/                          Category A
|   │   │
|   │   ├─ gutenburg/                 Project Gutenberg plain-text files
|   │   │   └─ standard_ebooks/       Standard Ebooks EPUB/TXT
|   │   └─ educational/               Category B
|   │       │
|   │       ├─ openstax/
|   │       └─ ck12/
|   └─ reasoning/                     Category C
|       │
|       ├─ gsm8k_train.jsonl
|       ├─ numinamath_train.jsonl
|       ├─ arc_challenge_train.jsonl
|       ├─ arc_easy_train.jsonl
|       └─ logiqa_train.jsonl
```

- └─ math_train.jsonl
- └─ conversations/ Category D
 - └─ dolly_15k.jsonl
 - └─ flan_train.jsonl
 - └─ oasst2_train.jsonl
- └─ technical_docs/ Category E
 - └─ wikipedia/
 - └─ extracted/ wikiextractor output
 - └─ filtered/ CS/math/science articles only ← use this
 - └─ official_docs/
 - └─ python_docs/
 - └─ pytorch_docs/
 - └─ hf_docs/
- └─ articles/ Category F
 - └─ arxiv/
- └─ research_papers/ Category G
 - └─ arxiv/ CC-BY papers only
 - └─ acl_anthology/
- └─ synthetic/ Category H (COC-generated)
 - └─ (populated by dataset/generator.py)
- └─ cognition/ Category I
 - └─ pg_psychology/ PD psychology books
 - └─ openstax_psychology/
- └─ retrieval/ Category J
 - └─ msmarco_train.jsonl
 - └─ nq_train.jsonl
 - └─ triviaqa_train.jsonl
- └─ kg/ Category K
 - └─ conceptnet_train.jsonl
 - └─ dbpedia/
 - └─ wikidata/
- └─ holdout/ Category L – EVAL ONLY, never train
 - └─ arc_test.jsonl
 - └─ mmlu_test.jsonl
 - └─ hellaswag_val.jsonl
- └─ cleaned/ STAGE 3+4 – normalised + cleaned UTF-8 text
 - └─ (same subdirs as raw/)
- └─ deduplicated/ STAGE 5 – after within-source dedup
 - └─ (same subdirs)
- └─ scored/ STAGE 6 – paragraph score metadata
 - └─ <source_id>_scores.jsonl
- └─ review_queue/ STAGE 7 – human review items
 - └─ pending.jsonl Items awaiting review

		approved.jsonl	Operator-approved items
		rejected.jsonl	Operator-rejected items
		approved/	STAGE 8 – release-eligible text
			(same subdirs as raw/)
		rejected/	Failed content – kept for audit, never used
			(same subdirs)
		synthetic/	Versioned COC synthetic data
			v1/
			v2/
		manifests/	Per-source manifests (JSON)
			<source_id>.json
		releases/	Assembled training releases
			v1/
			train.txt Training split (90%)
			val.txt Validation split (5%)
			test.txt Test split (5%) – holdout until eval
			manifest.json Full release metadata
			checksums.sha256 SHA-256 of all three splits
			LOCK Written by lock-release; prevents mutation
		governance_logs/	Audit JSONL daily shards
			audit_YYYYMMDD.jsonl
		archive/	Retired / recalled artifacts
			recalled/
			v1_recalled/ If v1 was recalled
			v1_incident.json Incident record

B.4 File Naming Conventions

Item	Convention	Example
Source ID	<category>-<short_name>-<YYYYMMDD>	A-gutenberg-20260701
Source raw dir	warehouse/raw/<category_dir>/<source_short>/	raw/books/gutenberg/
Source manifest	warehouse/manifests/<source_id>.json	manifests/A-gutenberg-20260701.json

Item	Convention	Example
Pipeline output	<code>warehouse/<stage>/<category_dir>/<source_id>.txt</code>	<code>cleaned/books/A-gutenberg-20260701.txt</code>
Score file	<code>warehouse/scored/<source_id>_scores.jsonl</code>	<code>scored/A-gutenberg-20260701_scores.jsonl</code>
Release dir	<code>warehouse/releases/v<N>/</code>	<code>releases/v1/</code>
Audit shard	<code>warehouse/governance_logs/audit_<YYYYMMDD>.jsonl</code>	<code>audit_20260701.jsonl</code>

B.5 Expected File Types and Sizes

Stage	Format	Approx. size (30B token release)
Raw archives	<code>.txt</code> , <code>.jsonl</code> , <code>.epub</code> , <code>.pdf</code> , <code>.xml.bz2</code>	~150 GB
Cleaned text	<code>.txt</code> (UTF-8, double-newline paragraphs)	~110 GB
		
Deduplicated	<code>.txt</code>	~65 GB
Score metadata	<code>.jsonl</code> (one JSON per paragraph)	~5 GB
Approved text	<code>.txt</code>	~55 GB
Release (tokenized)	<code>uint16 .bin</code> after tokenization	~60 GB
Manifests	<code>.json</code>	< 1 MB each
Audit logs	<code>.jsonl</code>	~2 GB/year

B.6 Warehouse Statistics Command

```
bash
python main.py corpus warehouse-stats
```

Output shows: tokens by category, GB used, % toward v1 (30B) and warehouse targets, completed releases.

C. Source Acquisition Manual

C.1 Source Lifecycle



C.2 How to Add a New Source — Complete Steps

Step 1: Discover and evaluate the source

Before registering, answer:

- What is the exact licence? (Check licence page, README, or header comment)
- Is the licence in `governance/license_rules.json`?
- Is the `licence_risk` ≤ 0.20 ? (Required for release v1)
- Is the content in English?
- Does the content match one of categories A-L?

Step 2: Download raw files

Place files in `D:/corpus_warehouse/raw/<category_dir>/<source_short_name>/`

Example for Project Gutenberg:

```
bash

# Create directory
mkdir -p "D:/corpus_warehouse/raw/books/gutenberg/"

# Rsync from PG mirror (recommended for bulk download):
rsync -av --del --include='*.txt' --exclude='*' \
  ftp@ftp.ibiblio.org::gutenberg/cache/epub/ \
  "D:/corpus_warehouse/raw/books/gutenberg/"
```

Step 3: Register the source

```
bash
```

```
python main.py corpus register-source A-gutenberg-v1 \
  --category A \
  --name "Project Gutenberg English PD Texts" \
  --url "https://gutenberg.org" \
  --licence "Public Domain" \
  --licence-id pd \
  --licence-risk 0.0 \
  --raw-path "D:/corpus_warehouse/raw/books/gutenberg/" \
  --raw-sha256 "" \
  --operator mpssp \
  --notes "Top English PD texts via rsync mirror"
```

```
# Output: ✓ Source registered: A-gutenberg-v1
# Effect: Entry added to governance/source_registry.json
# Audit: acquire → source_registered event written
```

What **register-source** writes:

- **governance/source_registry.json** — new source entry
- **governance/approval_log.jsonl** — acquisition event
- **D:/corpus_warehouse/governance_logs/audit_<today>.jsonl** — audit event

Step 4: Validate the source

After reviewing the source licence and content (spot-check 3–5% of files):

```
bash

python main.py corpus validate-source A-gutenberg-v1 --operator mpssp

# Output: ✓ VALIDATED: A-gutenberg-v1 Status: validated
# Effect: source.source_validated = True; status = "validated"
# Audit: validate → source_validated event written
```

To reject a source:

```
bash

python main.py corpus validate-source A-gutenberg-v1 \
  --reject --reason "OCR quality unacceptable for training" --operator mpssp
# Effect: status = "rejected"; source blocked from pipeline
```

Step 5: Run the pipeline

```
bash
```



```
# Run all stages at once:
python main.py corpus run-pipeline A-gutenberg-v1 \
  --stages clean,dedup,score \
  --operator mpssp \
  --verbose

# Or run stages individually:
python main.py corpus run-pipeline A-gutenberg-v1 --stages clean --verbose
python main.py corpus run-pipeline A-gutenberg-v1 --stages dedup --verbose
python main.py corpus run-pipeline A-gutenberg-v1 --stages score --verbose
```

What **run-pipeline** writes:

- `warehouse/cleaned/<category>/<source_id>.txt` — after clean stage
- `warehouse/deduplicated/<category>/<source_id>.txt` — after dedup stage
- `warehouse/approved/<category>/<source_id>.txt` — auto-approved paragraphs
- `warehouse/review_queue/pending.jsonl` — borderline paragraphs
- `warehouse/rejected/<category>/<source_id>.txt` — auto-rejected paragraphs
- `warehouse/scored/<source_id>_scores.jsonl` — score metadata

Step 6: Review borderline items (if any)

```
bash

# Check if anything needs review:
python main.py corpus review --stats
# Output: Pending: 42000  Approved: 150000  Rejected: 5000

# Start review session (required if pending > 0):
python main.py corpus review --source A-gutenberg-v1 --operator mpssp

# Controls during review:
# [a] approve this item
# [r] reject (prompts for reason)
# [s] skip (leave in queue)
# [q] quit session
```

Step 7: Verify source approval

```
bash
```

```
# Check source status in registry:
python main.py corpus audit-report --source A-gutenberg-v1

# Check warehouse tokens:
python main.py corpus warehouse-stats

# Category A should now show > 0 tokens
```

C.3 How to Update a Source

Sources cannot be modified in-place (registry is append-only). To update a source:

```
bash

# 1. If the source needs re-cleaning (e.g. better cleaning algorithm found):
python main.py corpus run-pipeline A-gutenberg-v1 --stages clean --verbose
# Re-running clean overwrites warehouse/cleaned/ output for that source

# 2. If the source needs re-scoring (e.g. threshold changed):
python main.py corpus run-pipeline A-gutenberg-v1 --stages score --verbose

# 3. If the licence was wrong, reject and re-register with correct info:
python main.py corpus validate-source A-gutenberg-v1 \
    --reject --reason "Licence correction needed" --operator mpssp
# Then register the corrected version:
python main.py corpus register-source A-gutenberg-v1b \
    --licence "CC0 1.0" --licence-id cc0 --licence-risk 0.0 ...
```

C.4 How to Archive a Source

When a source is no longer needed but must be preserved for audit:

```
bash

# The audit log preserves all history automatically.
# To archive raw files from the SSD:
# 1. Move raw dir to warehouse/archive/
mv "D:/corpus_warehouse/raw/books/old_source/" \
    "D:/corpus_warehouse/archive/old_source_$(date +%Y%m%d)/"

# 2. Update registry status to "archived" (via source_registry.py):
python -c "
from corpus.source_registry import update_source
update_source('A-old-source-20250101', {'status': 'archived'})
print('Archived')
"
```

C.5 Source Acquisition Helper Script

The `scripts/acquire_zero_risk.py` script provides step-by-step instructions for every zero-risk source family:

```
bash

# List all available source families:
python scripts/acquire_zero_risk.py --list

# Get instructions for a specific source:
python scripts/acquire_zero_risk.py --source A-gutenberg
python scripts/acquire_zero_risk.py --source C-reasoning
python scripts/acquire_zero_risk.py --source G-arxiv

# Get instructions for a whole category:
python scripts/acquire_zero_risk.py --category A

# Get all acquisition instructions at once:
python scripts/acquire_zero_risk.py --all
```

What the script does: Prints the exact download commands, rsync/wget/Python code, and the `register-source` command to run after downloading. It does NOT download automatically — all acquisition is operator-confirmed.

D. Licensing and Governance Manual

D.1 The Zero-Risk-First Policy

COC v3 enforces a **zero-risk-first** corpus policy. The policy is codified in two places:

1. `governance/license_rules.json` — the risk matrix (18 licence types)
2. `config.py CORPUS_POLICY` — enforcement thresholds

Core rule: Release v1 only contains sources with `licence_risk ≤ 0.20`.

```
python
```

```
# config.py CORPUS_POLICY block:
CORPUS_POLICY = dict(
    max_licence_risk_v1          = 0.20,    # HARD CEILING for release v1
    max_licence_risk_warehouse = 0.50,    # Warehouse-only ceiling (NC etc.)
    require_explicit_licence    = True,    # Reject sources with no licence
    require_source_validated    = True,    # Block pipeline on unvalidated sources
    synthetic_proportion_cap    = 0.033,   # Max 3.3% synthetic in any release
    holdout_categories          = ["L"],    # Never in any training split
    zero_risk_licence_ids       = [
        "pd", "cc0", "mit", "apache2", "bsd3", "psf",
        "cc-by-4", "cc-by-3", "cc-by-sa-4", "cc-by-sa-3", "gp12",
    ],
    warehouse_only_licence_ids = [
        "cc-by-nc-4", "cc-by-nc-sa", "cc-by-nd",
    ],
    hard_reject_licence_ids     = ["ars", "tos-violation", "unknown"],
)
```

D.2 Approved Licences for Release v1

Licence	Risk	v1 Release	Notes
Public Domain / pre-1927	0.0	✓ YES	No restrictions
CC0 1.0	0.0	✓ YES	Explicit PD dedication
MIT	0.05	✓ YES	Attribute in manifest
Apache 2.0	0.05	✓ YES	Attribute in manifest
BSD 3-Clause	0.05	✓ YES	Attribute in manifest
PSF License	0.05	✓ YES	Python stdlib docs
CC-BY 4.0	0.10	✓ YES	Attribute in manifest
CC-BY 3.0	0.10	✓ YES	Attribute in manifest
CC-BY-SA 4.0	0.20	✓ YES	Document SA; note in manifest
CC-BY-SA 3.0	0.20	✓ YES	Document SA; note in manifest
GPL v2 (prose only)	0.30	✓ prose only	Not code output

D.3 Warehouse-Only Licences (Not in Release v1)

Licence	Risk	Warehouse	Release v1	Reason
CC-BY-NC 4.0	0.40	✔ YES	✗ NO	Non-commercial restriction
CC-BY-NC-SA 4.0	0.45	✔ YES	✗ NO	NC + SA
CC-BY-ND	0.50	Review	✗ NO	ND clause — pending legal review

D.4 Hard Rejects — Never Included Anywhere

Source type	Reason
Post-1927 copyrighted books	Copyright not expired
News agency content (Reuters, AP, BBC, Bloomberg)	ToS prohibits training use
AWS/Azure/GCP proprietary docs	ToS prohibits
GPT-3.5/4/ChatGPT/Claude/Gemini outputs	OpenAI ToS + licence risk
Social media (Twitter/X, Reddit, Facebook)	ToS + licence unknown
SEO content farms	Quality reject + licence unknown
Z-Library / Anna's Archive / SciHub	Legally compromised
Any source with unknown provenance	Licence unknown → risk 0.85

D.5 How Licence Rules Are Enforced

```
Source registration
  ↓
licence_id provided → lookup in governance/license_rules.json
  ↓
licence_risk assigned automatically from rules
  ↓
If licence_risk > CORPUS_POLICY.max_licence_risk_v1 (0.20):
  → Source flagged warehouse-only in registry
  → run-pipeline proceeds but release builder REJECTS the source
  → manifest will NOT contain this source
  ↓
If licence_risk > CORPUS_POLICY.max_licence_risk_warehouse (0.50):
  → Source auto-rejected at validation stage
```

```
bash
```

```
python -c "  
import json  
rules = json.load(open('governance/license_rules.json'))  
for r in rules['rules']:  
    if r['licence_id'] == 'cc-by-4':  
        print(r)  
"
```

D.6 How the Approval Log Works

`governance/approval_log.jsonl` is an **append-only JSONL file committed to git**. Every approval decision is recorded here permanently.

Event structure:

```
json
```

```
{  
  "ts": "2026-07-01T14:22:00Z",  
  "ts_unix": 1751375120.0,  
  "stage": "validate",  
  "source_id": "A-gutenberg-v1",  
  "action": "source_validated",  
  "result": "ok",  
  "operator": "mpssp",  
  "hash": null,  
  "details": {"reason": "Reviewed PD status - all texts pre-1927"}  
}
```

To read the approval log:

```
bash
```

```
python main.py corpus audit-report  
python main.py corpus audit-report --source A-gutenberg-v1  
python main.py corpus audit-report --from 2026-07-01 --to 2026-07-31
```

D.7 Source Registry Schema

`governance/source_registry.json` is the **single source of truth** for all registered sources:

```
json
```

```

{
  "registry_version": "1.0",
  "coc_version": "v3",
  "last_updated": "2026-07-01T14:22:00",
  "registry_hash": "sha256:abc123...",
  "sources": [
    {
      "source_id": "A-gutenberg-v1",
      "name": "Project Gutenberg English PD Texts",
      "url": "https://gutenberg.org",
      "category": "A",
      "licence": "Public Domain",
      "licence_id": "pd",
      "licence_risk": 0.0,
      "acquisition_date": "2026-07-01",
      "acquired_by": "mpssp",
      "raw_path": "D:/corpus_warehouse/raw/books/gutenberg/",
      "raw_sha256": "",
      "source_validated": true,
      "validation_date": "2026-07-01",
      "validated_by": "mpssp",
      "status": "approved",
      "approved_paragraphs": 5900000,
      "approved_tokens_est": 12000000000,
      "approval_date": "2026-07-15",
      "approved_by": "mpssp",
      "in_release": ["v1"],
      "notes": "Top English PD texts"
    }
  ]
}

```

Registry hash: Automatically recomputed on every save. SHA-256 of the sources list. Changing any source record changes the registry hash — detectable via audit.

E. Corpus Quality System

E.1 Quality Score Formula

```

quality_score = word_count_score    × 0.20
                + lexical_richness  × 0.20
                + sentence_variety  × 0.15
                + printable_ratio    × 0.10
                + information_density × 0.15

```

```
+ pii_clean          × 0.10
+ non_repetitive     × 0.10
```

Score range: 0.0 - 1.0

Source file: `corpus/scorer.py::score_quality()`

E.2 Sub-Score Definitions

Sub-score	How calculated	Target range
<code>word_count_score</code>	<code>1.0</code> for 20–500 words; <code>0.5</code> for 8–20; <code>0.0</code> for < 8	1.0 for 20–500 words
<code>lexical_richness</code>	<code>unique_words / total_words / 0.5</code> (capped at 1.0)	> 0.6 (diverse vocab)
<code>sentence_variety</code>	<code>1.0</code> if avg 10–30 words/sentence	1.0 for technical prose
<code>printable_ratio</code>	<code>printable_chars / total_chars</code>	> 0.95 for clean text
<code>information_density</code>	Domain keyword density per category	Varies by category
<code>pii_clean</code>	<code>1.0</code> if no PII detected; <code>0.0</code> if PII present	Always 1.0 after redaction
<code>non_repetitive</code>	<code>1.0</code> if top-3 words < 30% of total	> 0.8 for quality text

E.3 Category Score

Category score measures how well a paragraph serves its target category's training purpose:

Category	Signal vocabulary	Implementation
A (Books)	Base quality — uses quality_score	<code>0.8</code> constant (prose quality sufficient)
B (Educational)	"explain", "learn", "concept", "step", "example"	<code>_vocab_density()</code>
C (Reasoning)	"therefore", "if", "then", "proof", "conclude"	Combined reasoning + technical
D (Conversations)	QA structure, <code><user></code> tag presence	Instruction vocab + format check
E (Technical)	"algorithm", "function", "model", "vector", "API"	Technical vocab density
F (Articles)	Long prose baseline	<code>0.7</code> constant
G (Research)	Technical + reasoning combined	50/50 blend
H (Synthetic)	Scored at generation time	<code>0.8</code> constant
I (Cognition)	"emotion", "cognition", "memory", "motivation"	Emotion + teaching vocab
J (Retrieval)	"according to", "passage", "evidence", "source"	Retrieval vocab
K (KG)	"is a", "consists of", "entity", "relation"	KG vocab

E.4 Risk Score

Risk score is **looked up from the source registry**, not computed from text:

```
python

# corpus/scorer.py::score_risk(source_id)
record = get_source(source_id)
return float(record.get("licence_risk", 0.8))
# Returns 0.8 (high risk) if source not found
```

E.5 Decision Thresholds

quality_score	risk_score	Decision	Destination
≥ 0.70	≤ 0.20	Auto-approve	<code>warehouse/approved/</code>

quality_score	risk_score	Decision	Destination
0.45 - 0.70	0.20 - 0.50	Human review	<code>warehouse/review_queue/pending.jsonl</code>
< 0.45	any	Auto-reject	<code>warehouse/rejected/</code>
any	> 0.50	Auto-reject	<code>warehouse/rejected/</code>
(synthetic, any)	≤ 0.20	Human review	<code>warehouse/review_queue/pending.jsonl</code>
(synthetic)	< 0.70 quality	Auto-reject	<code>warehouse/rejected/</code>

E.6 How to Interpret Scores

Good paragraph (auto-approved):

```
json
{"quality_score": 0.76, "category_score": 0.82, "risk_score": 0.0, "decision":
```

Borderline paragraph (human review):

```
json
{"quality_score": 0.51, "category_score": 0.68, "risk_score": 0.10, "decision":
```

→ Open review session, read the text, decide approve/reject.

Bad paragraph (auto-rejected):

```
json
{"quality_score": 0.31, "category_score": 0.20, "risk_score": 0.0, "decision":
```

→ No action needed. Logged to `warehouse/rejected/`.

E.7 Deduplication Thresholds

Level	Method	Threshold	Scope
Within-source	Cosine similarity	0.92	Single source family
Cross-source	MinHash Jaccard	0.85	All source families
Release leakage	Exact SHA-256	1.0	$\text{train} \cap \text{val}$, $\text{train} \cap \text{test}$

Config keys (adjustable):

```
python
```

```
CORPUS["dedup_within_threshold"] = 0.92
CORPUS["dedup_near_threshold"]   = 0.85
CORPUS["dedup_exact_threshold"]  = 1.0
```

E.8 Release Quality Gates

Before `build-release` completes, all of the following must pass:

```
✓ All sources in release have status = "approved"
✓ All synthetic sources (Category H) have 100% human-reviewed items
✓ Cross-source deduplication has been run
✓ Leakage check: train n val = 0 exact matches
✓ Leakage check: train n test = 0 exact matches
✓ No source has licence_risk > CORPUS_POLICY.max_licence_risk_v1 (0.20)
✓ Holdout categories (L) have zero paragraphs in any training split
```

These gates are checked by `corpus/release_builder.py::build()` and `release/verify.py::verify_release()`.

F. Release Engineering Manual

F.1 What a Release Is

A release is a **versioned, checksummed, locked, reproducible snapshot** of approved corpus data. It is the only data that may be passed to `train/train_model.py`.

A release has a strict lifecycle:

```
DRAFT → VERIFIED → SIGNED → LOCKED → TRAINING
```

Each transition is a human action with an audit event. No transition can be undone without creating a new version.

F.2 Release Build Commands

```
bash
```

```
# Step 1: Dry run (no files written – shows what would be built)
python main.py corpus build-release v1 \
  --categories A,B,C,D,E,F,G,H,I,J,K --dry-run
# Output: token counts per category, total estimated tokens

# Step 2: Build for real
python main.py corpus build-release v1 \
  --categories A,B,C,D,E,F,G,H,I,J,K \
  --token-budget 300000000000
# Output: train.txt, val.txt, test.txt, manifest.json, checksums.sha256
# Written to: D:/corpus_warehouse/releases/v1/
```

What **build-release** does:

1. Collects all approved paragraphs from each category
2. Shuffles with seed=42
3. Applies token budget cap
4. Splits 90/5/5 (train/val/test)
5. Stratifies val and test by category
6. Writes split files
7. Generates manifest.json with source breakdown
8. Generates checksums.sha256
9. Runs leakage check — fails if any overlap detected

F.3 Release Verification

```
bash

python main.py corpus verify-release v1
# Checks:
#   ✓ release_dir_exists
#   ✓ manifest_exists
#   ✓ manifest_valid_json
#   ✓ manifest_required_fields
#   ✓ split_ratios_sum (must = 1.0)
#   ✓ train_file_exists, val_file_exists, test_file_exists
#   ✓ train_checksum, val_checksum, test_checksum
#   ✓ release_signed (if require_signed=True)
#   ✓ release_locked (if require_locked=True)
#   ✓ no_train_val_leakage
#   ✓ no_train_test_leakage
```

```
bash
```

```
python main.py corpus sign-release v1 --operator mpssp
# What this does:
#   - Re-verifies all checksums
#   - Updates manifest.json: status = "signed", signed_by = "mpssp"
#   - Logs release approval to governance/approval_log.jsonl
#   - Logs release approval to audit JSONL shard
```

After signing, the manifest cannot be changed without creating a new version.

F.5 Release Locking

```
bash
```

```
python main.py lock-release v1 \
  --operator mpssp \
  --notes "Phase 1 pre-training - 30B zero-risk corpus"

# What this writes:
#   D:/corpus_warehouse/releases/v1/LOCK
#   Contents: {
#     "release_id": "v1",
#     "locked_at": "2026-09-01T10:00:00",
#     "locked_by": "mpssp",
#     "notes": "Phase 1 pre-training...",
#     "manifest_hash": "sha256:...",
#     "split_hashes": {"train": "sha256:...", "val": "...", "test": "..."},
#     "checksums_hash": "sha256:..."
#   }
```

After locking, any modification to `train.txt`, `val.txt`, `test.txt`, `manifest.json`, or `checksums.sha256` is detected by the resume guard before training and causes an abort.

F.6 Release Manifest Schema

Full schema documented in `docs/CORPUS_MASTER.md §9`. Key fields:

```
json
```

```
{
  "release_id": "v1",
  "status": "signed",
  "total_tokens_estimate": 30000000000,
  "split_ratios": [0.90, 0.05, 0.05],
  "shuffle_seed": 42,
  "zero_risk_policy": true,
  "max_licence_risk_included": 0.20,
  "quality_gate": {
    "min_quality_score": 0.45,
    "max_risk_score": 0.20,
    "all_sources_validated": true,
    "leakage_check_passed": true
  },
  "checksums": {
    "train": "sha256:...",
    "val": "sha256:...",
    "test": "sha256:..."
  }
}
```

F.7 Release Recall and Archive

If a release must be recalled (bad data discovered after training has started):

```
bash
```

```
# Step 1: Stop training (Ctrl+C)

# Step 2: Break the lock (admin action – permanently audited)
python main.py unlock-release v1 \
  --operator mpssp \
  --reason "Source A-bad-source-v1 contained copyright content – recall require

# Step 3: Reject the offending source
python main.py corpus validate-source A-bad-source-v1 \
  --reject --reason "Copyright content found – remove from warehouse" --operator

# Step 4: Remove bad source from warehouse
rm -rf "D:/corpus_warehouse/raw/books/bad_source/"
rm -rf "D:/corpus_warehouse/approved/books/A-bad-source-v1.txt"

# Step 5: Rebuild as v2
python main.py corpus build-release v2 \
  --categories A,B,C,D,E,F,G,H,I,J,K --token-budget 300000000000
python main.py corpus verify-release v2
python main.py corpus sign-release v2 --operator mpssp
python main.py lock-release v2 --operator mpssp --notes "v1 recall recovery"

# Step 6: Resume training from v2
python main.py resume-verify v2 --fresh --operator mpssp
python main.py train-model D:/corpus_warehouse/releases/v2/train.txt
```

The original v1 release artifacts are moved to `warehouse/archive/recalled/` — never deleted.

F.8 Release Versioning

Release	Status	Purpose
v1	First release	30B zero-risk tokens, Phase 1 pre-training
v2	If v1 is recalled or expanded	Built after v1 issues resolved
v3+	Future expansions	After warehouse grows with new zero-risk sources

List all releases:

```
bash

python main.py corpus list-releases
```

G. Training Governance Manual

G.1 The Four Training Control Modules

Module	File	Purpose
Training Ledger	train/training_ledger.py	Append-only record of every session
Shard Tracker	train/shard_tracker.py	Tracks exact shard consumption
Resume Guard	train/resume_guard.py	Pre-training integrity gate (8 checks)
Provenance	train/provenance.py	Permanent model-to-data lineage record

G.2 Training Ledger

File: var/checkpoints/training_ledger.jsonl

Format: Append-only JSONL — one JSON object per session open/close event

Never overwrites historical records

Each session produces two events:

1. **Open event** (written at session start)
2. **Close event** (written at session end, including interruptions)

Key fields:

```
json
{
  "run_id": "run_20260901_080000",
  "release_id": "v1",
  "release_hash": "sha256:abc123...",
  "run_type": "pretrain",
  "start_step": 0,
  "end_step": 10000,
  "global_step": 10000,
  "tokens_session": 2621440000,
  "tokens_total": 2621440000,
  "start_ts": "2026-09-01T08:00:00",
  "end_ts": "2026-09-01T23:59:00",
  "checkpoint_path": "var/checkpoints/model_700m.pt",
  "optimizer_hash": "a1b2c3d4",
  "resume_count": 0,
  "status": "completed"
}
```


Query the ledger:

```
bash

python main.py training-ledger
# or:
python main.py corpus training-ledger
```

What it answers:

- What run IDs have occurred?
- What release was used in each run?
- What step range was covered?
- What checkpoint was produced?
- How many tokens were consumed?

G.3 Shard Tracker

File: `var/checkpoints/shard_tracker.json`

Format: JSON (atomic write — temp file + rename prevents corruption)

Purpose: Divide training corpus into 10M-token shards and track consumption

Shard states:

- `pending` — not yet consumed
- `in_progress` — currently being trained on (reset to `pending` on interruption)
- `completed` — fully consumed; will never be re-used

How duplicate training is prevented:

1. Shard tracker is initialised with all shards as `pending`
2. Before starting a shard, its hash is verified against the release manifest hash
3. If the release manifest hash has changed since initialisation → **abort** (release was modified)
4. On Ctrl+C: current shard reset from `in_progress` → `pending`
5. On session resume: `get_next_shard()` returns the first `pending` shard (not the interrupted one, since it was reset)
6. `completed` shards are never returned by `get_next_shard()`

View shard status:

```
bash
```

```
python main.py shard-status
# or:
python main.py corpus shard-status
```

Output shows:

```
COC v3 - Shard Tracker Status
Release:      v1  [abc123def456...]
Total shards: 30
Completed:    5  (16.7%)
In progress:  0
Pending:      25
Tokens done:  50,000,000 / 300,000,000 (16.7%)
```

G.4 Resume Guard (8-Point Check)

File: `train/resume_guard.py`

When called: Before every training session (fresh start or resume)

The guard runs 8 sequential checks. All must pass before training may proceed:

Check #	Name	What it verifies
1	<code>release_exists</code>	Release directory and manifest.json exist
2	<code>release_signed</code>	<code>manifest.status == "signed"</code>
3	<code>release_locked</code>	LOCK file exists
4	<code>release_checksums</code>	SHA-256 of train/val/test matches manifest
5	<code>manifest_hash_tracker</code>	Shard tracker initialised for same manifest hash
6	<code>no_stuck_shards</code>	No shards in <code>in_progress</code> state (auto-resets if found)
7	<code>checkpoint</code>	Checkpoint file exists and loads cleanly (if resuming)
8	<code>ledger</code>	Ledger shows same release_id as current release

On any failure: `ResumeGuardError` is raised, training does not start, failure is logged to audit.

```
bash
```

```
# Run guard before first session:
python main.py resume-verify v1 --fresh --operator mpssp

# Run guard before resumed session:
python main.py resume-verify v1 --operator mpssp

# Expected output on pass:
# [resume_guard] Verifying release 'v1'...
#   ✓ release_exists
#   ✓ release_signed
#   ✓ release_locked
#   ✓ release_checksums
#   ✓ manifest_hash_tracker
#   ✓ no_stuck_shards
#   ✓ checkpoint
#   ✓ ledger
#   → All checks passed. Training may proceed.
```

G.5 Training Provenance

File: `var/checkpoints/provenance.json`

Format: JSON, append-per-run (never overwrites)

The provenance record permanently maps every model checkpoint to the exact corpus release and training sessions that produced it.

Per-run entry includes:

- `release_id` and `release_hash`
- Step range, global step, tokens consumed
- Train loss, val loss, perplexity
- Duration (hours), tokens/sec
- Checkpoint path + SHA-256
- Source breakdown by category (from release manifest)
- Evaluation results (if eval was run)

View provenance:

```
bash

python main.py provenance-report
# or:
python main.py corpus provenance-report
```

G.6 Token Accounting

The system tracks tokens at three levels:

Level	Source	Command
Per-session tokens	<code>training_ledger.jsonl</code> → <code>tokens_session</code>	<code>python main.py training-ledger</code>
Cumulative tokens	<code>training_ledger.jsonl</code> → <code>tokens_total</code>	<code>python main.py training-ledger</code>
Shard-level tokens	<code>shard_tracker.json</code> → <code>token_count</code> per shard	<code>python main.py shard-status</code>

To get total tokens trained to date:

```
bash
python -c "from train.training_ledger import get_total_tokens; print(f'{get_tot
```

G.7 Checkpoint Lineage

Every checkpoint is linked to:

1. The **release** that was being trained on (via `training_ledger.jsonl`)
2. The **shard** that completed when the checkpoint was written (via `shard_tracker.json`)
3. The **session** that produced it (via `training_ledger.jsonl`)
4. The **evaluation scores** at checkpoint time (via `provenance.json`)

To trace a checkpoint:

```
bash
python -c "
from train.provenance import get_checkpoint_lineage
lineage = get_checkpoint_lineage('var/checkpoints/model_700m.pt')
for entry in lineage:
    print(f'Run: {entry["run_id"]} Release: {entry["release_id"]} Steps:
"
```

G.8 Recovery After Crash or Power Loss

```
bash
```

```
# 1. Check what state things are in:
python main.py training-ledger          # last session status
python main.py shard-status             # any stuck shards?
python main.py resume-verify v1         # will auto-reset stuck shards + verify a

# 2. If checkpoint integrity is questioned:
python -c "
import torch
c = torch.load('var/checkpoints/model_700m.pt', map_location='cpu', weights_only=True)
print(f'Step: {c.get(\"step\", \"unknown\")}')
print(f'Keys: {list(c.keys())}')
"

# 3. If checkpoint is corrupted (load fails):
# → Fall back to model_700m_best.pt (last best val-loss checkpoint)
# → Update training to start from that step
python main.py resume-verify v1 --operator mpssp # guard will flag checkpoint

# 4. If shard_tracker.json is corrupted:
# → Delete it – training will reinitialise (loses shard progress but not model)
rm var/checkpoints/shard_tracker.json
python main.py resume-verify v1 --fresh --operator mpssp
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt

# 5. If training_ledger.jsonl is corrupted:
# → It's append-only JSONL – partial corruption only affects the corrupted line
# → The file can be repaired by removing the last incomplete JSON line:
python -c "
lines = open('var/checkpoints/training_ledger.jsonl').readlines()
import json
valid = [l for l in lines if l.strip()]
for l in valid:
    try: json.loads(l)
    except: valid.remove(l); print(f'Removed: {l[:60]}')
open('var/checkpoints/training_ledger.jsonl', 'w').writelines(valid)
"
```

H. Segmented Training Playbook

H.1 Why Segmented Training

COC pre-training takes ~260 days at 16h/day on the RTX 5060. Continuous training is impractical and unsafe. Segmented training:

- Allows daily cooldown periods (thermal and electrical)
- Checkpoints every 1000 steps (saves at worst 1000 steps of progress)
- Makes training resumable after power loss, OS update, or shutdown
- Allows monitoring between sessions (check losses, validate at milestones)

H.2 The 16-Hour Session Pattern

Recommended daily schedule:

```
06:00 → Run resume guard (verify release + checkpoint integrity)
06:05 → Start training session
...training runs for 16 hours...
22:05 → Session completes or is stopped with Ctrl+C
22:10 → Check session summary (training-ledger)
22:15 → System cools down
06:00 → Repeat next day
```

GPU temperature management: Keep GPU below 85°C. The observability system logs `gpu_temp_c` every 2 seconds. If temp exceeds 80°C consistently, consider reducing batch size or adding cooling.

H.3 Starting the First Session

```
bash

# Pre-flight checklist:
# □ CORPUS_WAREHOUSE_DIR set in config.py
# □ Release v1 built, signed, locked
# □ Tokenizer trained (var/tokenizer/coc_tokenizer.model exists)
# □ Sufficient free VRAM (< 1.5 GB other processes)

# Step 1: Run resume guard (--fresh = not a resume)
python main.py resume-verify v1 --fresh --operator mpssp
# Expected: 8 checks, all ✓

# Step 2: Start training
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt

# What you'll see:
# Step 0100 | loss 8.2134 | val_loss -- | perp -- | gnrm 0.82 | lr 3.00e-5 | t
# Step 0200 | loss 7.8901 | val_loss -- | perp -- | gnrm 0.74 | lr 6.00e-5 | t
# ...
# Step 0500 | loss 7.1234 | val_loss 7.4521 | perp 1722.1 | gnrm 0.61 | lr 1.5
```

H.4 Stopping Safely

```
bash

# To stop after current step completes:
Ctrl+C

# What happens:
# 1. Training loop catches KeyboardInterrupt
# 2. Current shard is reset to "pending" in shard_tracker.json
# 3. Checkpoint is saved to var/checkpoints/model_700m.pt
# 4. Training ledger close event written with status="interrupted"
# 5. Console shows: "Saving checkpoint before exit..."

# Verify stop was clean:
python main.py training-ledger
# Last entry should show: status=interrupted, checkpoint_path=var/checkpoints/m
```

H.5 Resuming Training

```
bash

# Step 1: Run resume guard (always required before resuming)
python main.py resume-verify v1 --operator mpssp
# Guard will:
# - Verify release v1 LOCK file integrity
# - Verify manifest hash matches shard tracker
# - Auto-reset any stuck in_progress shards
# - Verify checkpoint loads correctly
# Expected: all 8 checks ✓

# Step 2: Resume
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt --resume
# What you'll see:
# Resuming from step 10000...
# Step 10001 | loss 5.8234 | ...
```

H.6 Monitoring Between Sessions

```
bash
```

```
# After each session, run these checks:

# 1. Session summary
python main.py training-ledger
# Shows: run_id, release, steps covered, tokens consumed, duration

# 2. Shard progress
python main.py shard-status
# Shows: completed/pending/in-progress shards, % complete

# 3. Training metrics
tail -20 var/checkpoints/train_log.jsonl | python -c "
import json, sys
for line in sys.stdin:
    d = json.loads(line)
    print(f'Step {d["step"]:>6} | loss {d["train_loss"]:.4f} | val {d.get(\
"

# 4. Full provenance
python main.py provenance-report
```

H.7 Phase Gates

Run evaluation at scheduled milestones:

```
bash

# After 30K steps (~26 days):
python main.py eval
python main.py gate 1
# Expected: perplexity < 100, val_loss < 5.0

# After 60K steps (~52 days):
python main.py eval
python main.py gate 2
# Expected: perplexity < 50, val_loss < 4.0

# After 100K steps (~87 days):
python main.py eval
python main.py gate 3
# Expected: perplexity < 20, val_loss < 3.0
```


H.8 What to Check if Training Looks Wrong

Symptom	Likely cause	Fix
<code>train_loss</code> not decreasing after 5K steps	LR too low or too high	Check <code>TRAIN.lr</code> in config; try <code>1e-4</code> to <code>3e-4</code>
<code>grad_norm</code> > 5.0	LR too high or bad batch	Lower LR; check for corrupted paragraphs in corpus
<code>val_loss</code> increasing while <code>train_loss</code> falls	Overfitting or data quality	Check if review queue had too many borderline approvals
<code>tokens_per_sec</code> < 1000	VRAM pressure or thermal throttling	Reduce batch; check GPU temp
Training crashes with CUDA OOM	VRAM too low	<code>TRAIN.batch = 1</code> , <code>gradient_checkpointing = True</code>

I. Corpus Target Strategy

I.1 Warehouse vs Release — Size Targets

Target	Size	Timeline
Release v1 (first training release)	30B tokens	~8-10 weeks of acquisition
Warehouse zero-risk ceiling	~35B tokens	Full PD + CC0 + explicit-permissive
Warehouse permissive-first ceiling	~80B tokens	With warehouse-only NC sources
Originally stated 150B target	NOT achievable on zero-risk	Would require unknown-licence crawls

This is stated explicitly and not obscured. The 150B target requires moving beyond zero-risk sources, which violates the COC corpus policy for release v1. The 35B ceiling is the honest maximum for a fully-auditable, zero-risk corpus.

I.2 Why Warehouse > Release

The warehouse holds more data than any single release because:

1. Warehouse-only sources (CC-BY-NC, CC-BY-ND) may be added for future internal-use experiments
2. Older release versions are preserved for reproducibility
3. Synthetic data grows over time as COC is used in production
4. New zero-risk sources may become available

The release is always a **curated snapshot** of the highest-quality warehouse content, not the full warehouse.

I.3 Why Quality > Volume

A 30B token curated corpus of PD books, peer-reviewed research, and official documentation will produce a better 700M model than a 150B token dump of web crawl data.

The Chinchilla ratio for a 700M model is ~ 10 tokens/parameter ≈ 7 B tokens optimal. COC v1 release is 30B tokens — $4\times$ Chinchilla optimal. This provides multi-epoch training signal from high-quality sources rather than single-pass training on noisy web text.

I.4 Corpus Expansion Path

```
Release v1 (30B) → validate model quality at 30K/60K/100K steps
    ↓
Add new zero-risk sources to warehouse (PG long-tail, more ArXiv CC-BY)
    ↓
Release v2 (35B max zero-risk ceiling)
    ↓
If CoC becomes non-commercial confirmed, add CC-BY-NC warehouse sources
    ↓
Release v3 (~50B with warehouse-only sources)
```

I.5 Category Token Targets (v1)

From `config.py CORPUS_POLICY.category_token_targets_v1`:

Category	Target	%	Rationale
A — Books	12B	40%	Long-form coherence is hardest to learn; PD books are the best source
E — Technical docs	4B	13%	Wikipedia CS/math + official docs anchor factual accuracy
C — Reasoning/STEM	3B	10%	Critical for chain-of-thought; all MIT/Apache licenced
G — Research papers	3B	10%	Highest-quality technical prose; CC-BY only
B — Educational	2B	7%	Structured explanation; OpenStax CC-BY
F — Articles	2B	7%	Sustained coherence; ArXiv intros + Wikinews
D — Conversations	2B	7%	Instruction-following; Dolly + FLAN + OASST2
I — Cognition	0.5B	2%	Supports 9-module cognition layer
J — Retrieval	0.5B	2%	Grounds retrieval-style responses
K — KG	0.5B	2%	Supports triple extraction training
H — Synthetic	0.5B	2%	COC-specific patterns (reviewed)
Total	30B	100%	

J. Complete CLI Reference (Updated)

┆ This section supersedes §9. All commands are alphabetical within groups.

J.1 Top-Level Commands

```
bash

# Working directory for all commands: cognitiveoc_v3/
python main.py <command> [options]
```

J.2 Corpus Subcommands

bash

```
python main.py corpus <subcommand> [options]
```


Subcommand	Purpose	Outputs
<code>audit-report [--source <id>] [--from <date>] [--to <date>]</code>	Generate audit report	Console
<code>build-release <ver> [--categories ...] [--token-budget N] [--dry-run]</code>	Assemble release	<code>releases/v<N>/</code>
<code>list-releases</code>	List all releases with status	Console
<code>lock-release <ver> --operator <you> [--notes "..."]</code>	Lock release	<code>releases/v<N>/LOCK</code>
<code>provenance-report</code>	Print training provenance	Console
<code>register-source <id> --category <A-L> --name --url --licence --licence-id --licence-risk --raw-path --operator</code>	Register new source	<code>governance/source_registry.json</code>
<code>resume-verify <ver> [--fresh] [--operator <you>]</code>	Pre-training guard	Console + audit log
<code>review [--source <id>] [--operator <you>] [--stats]</code>	Human review session	<code>warehouse/review_queue/</code>
<code>run-pipeline <id> --stages clean,dedup,score [--operator <you>] [--verbose]</code>	Run pipeline stages	<code>warehouse/cleaned/</code> , <code>deduplicated/</code> , <code>approved/</code>
<code>shard-status</code>	Print shard tracker	Console
<code>sign-release <ver> --operator <you></code>	Sign release	<code>manifest.json</code> updated
<code>training-ledger</code>	Print training ledger	Console
<code>unlock-release <ver> --operator --reason</code>	Admin: break lock	LOCK archived

Subcommand	Purpose	Outputs
<code>validate-source <id> [--reject] [--reason "..."] --operator <you></code>	Validate/reject source	<code>governance/source_registry.json</code>
<code>verify-release <ver></code>	Verify checksums + leakage	Console
<code>warehouse-stats</code>	Warehouse token + storage stats	Console

J.3 Acquisition Helper

```
bash
python scripts/acquire_zero_risk.py [--list] [--all] [--source <name>] [--category <cat>]
```

Option	Purpose
<code>--list</code>	List all available acquisition targets
<code>--source A-gutenberg</code>	Print acquisition instructions for Project Gutenberg
<code>--source C-reasoning</code>	Print instructions for reasoning datasets
<code>--source E-wikipedia</code>	Print instructions for Wikipedia dump
<code>--source G-arxiv</code>	Print instructions for ArXiv CC-BY papers
<code>--category A</code>	Print all Category A acquisition instructions
<code>--all</code>	Print all acquisition instructions

K. Full End-to-End Story

This section traces the **complete path** from a fresh environment to a trained, evaluated, running system.

Step 1 — Install

```
bash
```

```
# Python requirement: 3.10+
python --version

# Core dependencies:
pip install torch --index-url https://download.pytorch.org/whl/cu124
pip install PySide6 sentencepiece transformers accelerate
pip install datasets requests tqdm pdfminer.six ebooklib html2text mwparserfrom

# Optional but recommended:
pip install datasketch      # 5× faster MinHash deduplication
pip install pymupdf         # better PDF text extraction

# Verify:
python -c "import config, engine, corpus.cli, train.training_ledger; print('All
```

Step 2 — Configure SSD

```
python

# Edit cognitiveoc_v3/config.py line ~530:
CORPUS_WAREHOUSE_DIR = Path("D:/corpus_warehouse") # ← YOUR ACTUAL SSD PATH
```

```
bash

# Verify:
python main.py corpus warehouse-stats
# Expected: all categories 0 tokens, no errors

python main.py corpus audit-report
# Expected: "Total events: 0"

python main.py status
# Expected: all subsystems initialised
```

Step 3 — Start the System

```
bash

# Desktop app (full backend):
python main.py ui --desktop
# Expected: PySide6 window opens, Status: "Backend Ready"

# Or web server:
python main.py ui
# Expected: Server listening on http://127.0.0.1:8765
```


Step 4 — Get Acquisition Instructions

```
bash

python scripts/acquire_zero_risk.py --all
# Prints step-by-step download instructions for all 11 source families
```

Step 5 — Register and Acquire Source A (Project Gutenberg)

```
bash

# Register BEFORE downloading (establishes the record):
python main.py corpus register-source A-gutenberg-v1 \
  --category A \
  --name "Project Gutenberg English PD Texts" \
  --url "https://gutenberg.org" \
  --licence "Public Domain" --licence-id pd --licence-risk 0.0 \
  --raw-path "D:/corpus_warehouse/raw/books/gutenberg/" \
  --operator mpssp

# Download (takes hours for bulk):
rsync -av --del --include='*.txt' --exclude='*' \
  ftp@ftp.ibiblio.org::gutenberg/cache/epub/ \
  "D:/corpus_warehouse/raw/books/gutenberg/"
```

Step 6 — Validate Source

```
bash

# After reviewing a sample of the texts (spot-check 10 files):
python main.py corpus validate-source A-gutenberg-v1 --operator mpssp
# Expected: ✓ VALIDATED: A-gutenberg-v1
```

Step 7 — Pipeline the Source

```
bash

python main.py corpus run-pipeline A-gutenberg-v1 \
  --stages clean,dedup,score --operator mpssp --verbose

# Expected output:
# [clean] 8,500,000 paragraphs written to warehouse/cleaned/books/
# [dedup] 8,500,000 → 7,200,000 paragraphs (1,300,000 removed, 84.7% retained)
# [score] auto-approved: 5,900,000 human-queue: 420,000 auto-rejected: 880,0
```

Step 8 — Review Borderline Items

```
bash

python main.py corpus review --stats
# Expected: Pending: 420000 Approved: 5900000 Rejected: 880000

# If pending > 0, run review session:
python main.py corpus review --source A-gutenberg-v1 --operator mpssp
# [a]pprove [r]eject [s]kip [q]uit
```

Step 9 — Repeat for All Categories

Repeat Steps 5-8 for each source family. Priority order:

1. `A-gutenberg-v1` (biggest win, zero risk) — Week 1
2. `A-standard-ebooks-v1` — Week 1
3. `C-reasoning-hf-v1` (HuggingFace datasets — fast download) — Week 2
4. `D-instruction-hf-v1` (Dolly + FLAN + OASST2) — Week 2
5. `E-wikipedia-cs-v1` (large — takes time) — Week 2-3
6. `E-official-docs-v1` (Python/PyTorch/HF docs) — Week 3
7. `G-arxiv-ccby-v1` (CC-BY papers) — Week 3-4
8. `G-acl-anthology-v1` — Week 4
9. `B-openstax-v1` — Week 4
10. `JK-retrieval-kg` (retrieval + KG datasets) — Week 5
11. `I-cognition-v1` (PG psychology + OpenStax Psych) — Week 5

Generate synthetic (Category H) during live use of the system — accumulates automatically.

Step 10 — Check Progress

```
bash

python main.py corpus warehouse-stats
# Monitor:
# - Category A should be near 12B tokens
# - Total toward 30B target
```

Step 11 — Build the Release

```
bash
```

```
# When all categories have enough approved tokens:

# Dry run first:
python main.py corpus build-release v1 \
  --categories A,B,C,D,E,F,G,H,I,J,K --dry-run
# Expected: ~30B tokens total, breakdown by category

# Build for real:
python main.py corpus build-release v1 \
  --categories A,B,C,D,E,F,G,H,I,J,K \
  --token-budget 300000000000
# Expected: train.txt (~27B tok), val.txt (~1.5B), test.txt (~1.5B)
# Written to: D:/corpus_warehouse/releases/v1/
```

Step 12 — Verify and Sign

```
bash

# Verify (checksums + leakage):
python main.py corpus verify-release v1
# Expected: all checks ✓

# Sign (human approval):
python main.py corpus sign-release v1 --operator mpssp
# Expected: manifest.status = "signed"
```

Step 13 — Lock the Release

```
bash

python main.py lock-release v1 \
  --operator mpssp \
  --notes "COC v3 Phase 1 pre-training - 30B zero-risk corpus"
# Expected: LOCK file written to releases/v1/LOCK
# Expected: manifest_hash, split_hashes, checksums_hash recorded in LOCK
```

Step 14 — Train the Tokenizer

```
bash
```

```
python main.py train-tokenizer D:/corpus_warehouse/releases/v1/train.txt
# Expected: var/tokenizer/coc_tokenizer.model + coc_tokenizer.vocab
# Duration: ~2-6 hours depending on file size

# Verify fertility:
python main.py eval --tokenizer
# Expected: fertility ≥ 3.5 chars/token
```

Step 15 — Run Pre-Training (Session 1)

```
bash

# Pre-flight check:
python main.py resume-verify v1 --fresh --operator mpssp
# Expected: 8 checks, all ✓

# Start training:
python main.py train-model D:/corpus_warehouse/releases/v1/train.txt
# Training TUI shows: step, loss, val_loss, perplexity, grad_norm, lr, tok/s
# Checkpoint saved every 1000 steps to: var/checkpoints/model_700m.pt
```

Step 16 — Resume Training (Sessions 2+)

```
bash

# Next day:
python main.py resume-verify v1 --operator mpssp
# Expected: all ✓ including checkpoint integrity

python main.py train-model D:/corpus_warehouse/releases/v1/train.txt --resume
# Expected: "Resuming from step N..."
```

Step 17 — Monitor Progress

```
bash

python main.py training-ledger      # session history
python main.py shard-status        # shard completion %
python main.py provenance-report    # full provenance

# Live metrics:
tail -f var/checkpoints/train_log.jsonl
```

Step 18 — Evaluate at Milestones

```
bash
```

```
# At 30K, 60K, 100K steps:
python main.py eval
python main.py gate 1      # or 2 or 3

# Results logged to: var/checkpoints/provenance.json
```

Step 19 — Deploy

```
bash

# After training completes, start the full system:
python main.py ui --desktop
# The trained model is loaded automatically from var/checkpoints/model_700m.pt
# All 13 desktop panels are now fully functional with the trained model
```

Step 20 — Archive

```
bash

# After training is complete and the model is deployed:
# 1. Archive raw files to free SSD space:
mv "D:/corpus_warehouse/raw/" "D:/corpus_warehouse/archive/v1_raw/"

# 2. Keep: approved/, releases/v1/, manifests/, governance_logs/
# 3. Lock the training ledger (copy to provenance record):
python main.py provenance-report > var/checkpoints/final_provenance_v1.txt

# 4. Commit governance files to git:
git add governance/
git commit -m "COC v3: v1 release governance records"
```

L. Project Status Matrix

L.1 Fully Implemented and Verified

Subsystem	Module(s)	CLI Entry	Status
700M decoder (architecture)	models/transformer.py	—	✓ FROZEN
48K tokenizer	tokenizer/tokenizer.py	train- tokenizer	✓ FROZEN

Subsystem	Module(s)	CLI Entry	Status
13-encoder hub	<code>encoder/hub.py</code>	—	✓ FROZEN
9-module cognition layer	<code>cognition/cognition.py</code>	API toggle	✓ FROZEN
8-type memory system	<code>memory/memory.py</code>	<code>api/memory</code>	✓ FROZEN
Hybrid retrieval (BM25+BGE)	<code>retrieval/rag.py</code>	<code>ingest</code>	✓ FROZEN
SQLite KG (500K triples)	<code>knowledge/graph.py</code>	<code>api/kg</code>	✓ FROZEN
Chain-of-thought reasoning	<code>reasoning/reasoner.py</code>	auto	✓ FROZEN
Multi-loop research engine	<code>research/engine.py</code>	<code>api/research</code>	✓ FROZEN
Fact/citation validation	<code>validation/validator.py</code>	<code>api/validate</code>	✓ FROZEN
Async workflow engine	<code>workflow/workflow.py</code>	<code>api/workflows</code>	✓ FROZEN
2-tier guardrail framework	<code>safety/guardrails.py</code>	toggle API	✓ FROZEN
8-type dataset generator	<code>dataset/generator.py</code>	<code>generate-</code> <code>corpus</code>	✓ FROZEN
Training loop (700M)	<code>train/train_model.py</code>	<code>train-model</code>	✓ FROZEN
Eval suite (13 metrics)	<code>eval/run_suite.py</code>	<code>eval</code> / <code>gate</code>	✓ FROZEN
Web UI (18 endpoints, SSE)	<code>ui/app.py</code>	<code>ui</code>	✓ FROZEN
Desktop UI (13 panels)	<code>ui/desktop.py</code>	<code>ui --desktop</code>	✓ FROZEN

Subsystem	Module(s)	CLI Entry	Status
Observability (hardware+req)	<code>observability/metrics.py</code>	<code>api/metrics</code>	✓ FROZEN
Base data pipeline	<code>data/pipeline.py</code>	<code>prepare-corpus</code>	✓ FROZEN
Corpus pipeline (12 stages)	<code>corpus/</code>	<code>corpus *</code>	✓ ACTIVE
Source registry	<code>corpus/source_registry.py</code>	<code>corpus</code> <code>register-source</code>	✓ ACTIVE
15 category cleaners	<code>corpus/cleaner.py</code>	<code>corpus run-pipeline</code>	✓ ACTIVE
3-axis quality scorer	<code>corpus/scorer.py</code>	<code>corpus run-pipeline</code>	✓ ACTIVE
MinHash LSH cross-source dedup	<code>corpus/dedup.py</code>	<code>corpus run-pipeline</code>	✓ ACTIVE
Human review queue	<code>corpus/reviewer.py</code>	<code>corpus review</code>	✓ ACTIVE
Manifest generation+validation	<code>corpus/manifest.py</code>	<code>corpus verify-release</code>	✓ ACTIVE
Release builder	<code>corpus/release_builder.py</code>	<code>corpus build-release</code>	✓ ACTIVE
Warehouse manager	<code>corpus/warehouse.py</code>	<code>corpus</code> <code>warehouse-stats</code>	✓ ACTIVE
Corpus CLI (16 subcommands)	<code>corpus/cli.py</code>	<code>corpus *</code>	✓ ACTIVE
Governance audit interface	<code>governance/audit.py</code>	<code>corpus audit-report</code>	✓ ACTIVE
Licence rules (18 types)	<code>governance/license_rules.json</code>	—	✓ ACTIVE
Approval log	<code>governance/approval_log.jsonl</code>	<code>corpus audit-report</code>	✓ ACTIVE

Subsystem	Module(s)	CLI Entry	Status
Audit logger (dual-write)	<code>audit/logger.py</code>	auto	✓ ACTIVE
Audit reporter	<code>audit/reporter.py</code>	<code>corpus audit-report</code>	✓ ACTIVE
Release lock system	<code>release/lock.py</code>	<code>lock-release</code>	✓ ACTIVE
Release verification	<code>release/verify.py</code>	<code>corpus verify-release</code>	✓ ACTIVE
Training ledger	<code>train/training_ledger.py</code>	<code>training-ledger</code>	✓ ACTIVE
Shard tracker	<code>train/shard_tracker.py</code>	<code>shard-status</code>	✓ ACTIVE
Resume guard (8-point)	<code>train/resume_guard.py</code>	<code>resume-verify</code>	✓ ACTIVE
Training provenance	<code>train/provenance.py</code>	<code>provenance-report</code>	✓ ACTIVE
Acquisition helper script	<code>scripts/acquire_zero_risk.py</code>	direct	✓ ACTIVE
Zero-risk corpus policy	<code>config.py</code> <code>CORPUS_POLICY</code>	—	✓ ACTIVE
CORPUS_MASTER.md	<code>docs/CORPUS_MASTER.md</code>	—	✓ ACTIVE

L.2 Partially Implemented

Item	What exists	What is missing
Tokenizer training	Architecture + CLI command	Requires v1 release to be built
700M decoder pre-training	Full training loop	Requires tokenizer + release
SFT fine-tuning	Supported by <code>train_model.py</code>	Requires instruction dataset + pre-trained base

Item	What exists	What is missing
Desktop corpus panel	All other panels present	Corpus/warehouse stats panel not yet in <code>ui/desktop.py</code>
Desktop training monitor	Panel reads <code>train_log.jsonl</code>	Requires active training run

L.3 Planned (Not Yet Implemented)

Item	Planned for	Notes
ONNX export for NPU inference	After pre-training	Encoder hub already supports NPU; decoder export pending
SFT (Supervised Fine-Tuning)	After pre-training	Requires instruction data + base model
Release v2	After v1 training validates	Expand warehouse with more CC-BY papers
Desktop corpus panel	Next UI sprint	<code>warehouse-stats</code> + source registry browser
Continuous synthetic generation	Production deployment	Loop: use → generate → review → approve → retrain
Retrieval-augmented fine-tuning	After SFT	Fine-tune on retrieval-grounded outputs

L.4 Deferred

Item	Reason deferred
150B warehouse target	Not achievable on zero-risk policy; honest ceiling is 35B zero-risk / 80B permissive
Multi-GPU training	Single RTX 5060 is the target hardware; multi-GPU not planned
Distributed retrieval	SQLite retrieval sufficient for 500K triples; scale only if needed
Model quantisation	Defer until after training; quantise to 4-bit for production inference

All features documented · All CLI commands included · Operator-complete